Statistical and Neural Machine Translation

A Comprehensive Tutorial for AI (Fundamental) Class Students

Dr. Ye Kyaw Thu (ရဲကျော်သူ)

Visiting Professor, Core AI Research Team, AI Research Group, NECTEC, Thailand

Lab Leader, Language Understanding Lab., Myanmar

May, 2026

Duration: $\approx$ 12 hours | Reference books: Koehn (2008), Koehn (2017)

Table of Contents

What is Machine Translation?

Definition

Machine Translation (MT) is the use of software to automatically translate text or speech from one natural language (source) to another (target) without human intervention.

- One of the **oldest problems** in Artificial Intelligence
- Considered **AI-complete**: full understanding requires reasoning, world knowledge, and common sense
- Today, MT is a multi-billion-dollar industry (Google Translate, DeepL, ChatGPT, etc.)

Example

Myanmar example:
ကျွန်တော်သည် ကွန်ပျူတာသိပ္ပံဘာသာရပ်ကို လေ့လာနေသည်။
"I am studying Computer Science."

The Rosetta Stone: A Parallel Text

- Discovered by French soldiers in **Rosetta (Egypt) in 1799** during Napoleon's Egyptian Campaign
- The stone contains a priestly decree (**196 BC**) written in **three scripts**:
  1. Ancient Egyptian hieroglyphics (formal script)
  2. Demotic script (everyday Egyptian)
  3. Ancient Greek (language of the rulers)
- It is essentially the world's most famous **parallel text (bilingual corpus)**
- Modern MT systems use the same idea: *learn from translated text pairs*

Key Insight

The Rosetta Stone allowed humans to *decipher* a forgotten language. Statistical MT allows computers to *learn* translation from millions of such parallel sentences.

Jean-François Champollion (1790–1832)

- French scholar and **linguist**
- On **14 September 1822**, Champollion announced his breakthrough in deciphering ancient Egyptian hieroglyphics
- He used the **Rosetta Stone** as the key: matching known Greek text against Egyptian scripts
- His method was essentially **alignment-based translation** — the same principle used in IBM Word Alignment Models (1993)!
- Published in *Lettre à M. Dacier* (1822)

> *"It is a writing system of signs, at the same time figurative, symbolic, and phonetic, in the same text, the same sentence, the same word."*
>
> — Champollion, 1822

Connection to MT

Champollion's insight: a **known translation** can reveal the structure of an unknown language. This is the *statistical alignment* idea, 170 years before IBM models.

The Warren Weaver Memorandum (1947)

- **Warren Weaver** (Rockefeller Foundation) wrote a famous memo in 1947 proposing the use of computers for translation
- Inspired by success of **code-breaking** in World War II (e.g., Enigma)
- Key ideas in the memo:
  1. Translation as a **cryptographic** problem
  2. Use of **statistical analysis** of language
  3. Universal underlying **meaning** across languages
- The memo said: *"When I look at an article in Russian, I say: This is really written in English, but it has been coded in some strange symbols."*

Weaver's Vision (1947)

$\rightarrow$ Computers can decode foreign text

$\rightarrow$ Statistical patterns reveal meaning

$\rightarrow$ The **noisy channel model** idea!

This vision became reality 45 years later with **IBM models (1993)**.

Georgetown–IBM Experiment (1954)

- **January 7, 1954**: First public demonstration of MT at **IBM headquarters, New York**
- Developed by Georgetown University and IBM
- The system translated **60 Russian sentences into English**
- Used only:
  - **6 grammar rules**
  - **250 vocabulary items**
- Covered topics: politics, law, mathematics, chemistry, metallurgy

Optimistic Predictions (1954)

Researchers predicted MT would be **"solved in 3–5 years"**. This optimism led to massive Cold War funding from both the USA and USSR for MT research.

Reality Check

The problem proved far harder than expected. True high-quality MT was not achieved until the **2010s** with neural methods.

Cold War Era: USA & USSR MT Research

United States (1950s–1960s):

- Funded by **NSF, DARPA, CIA, DoD**
- Goal: translate Russian scientific literature quickly
- Many universities built RBMT systems
- Key projects: SYSTRAN, Mark II

Soviet Union:

- Parallel development in USSR
- Goal: translate US military and scientific documents
- Work at Moscow State University and other institutes

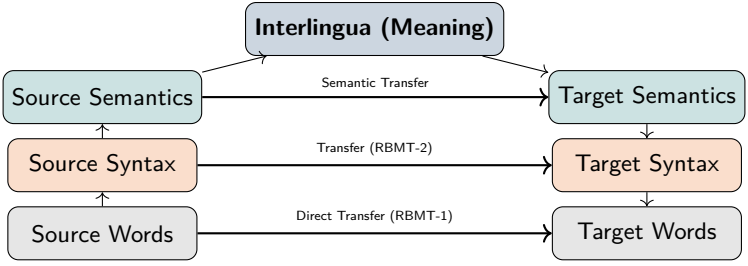
ALPAC Report (1966)

The **Automatic Language Processing Advisory Committee** (ALPAC) report concluded that:

- MT was **twice as expensive** as human translation
- **Twice as slow**
- Of **lower quality**

$\Rightarrow$ US government **stopped funding MT** research for almost a decade.
$\Rightarrow$ Known as the "**MT winter**"

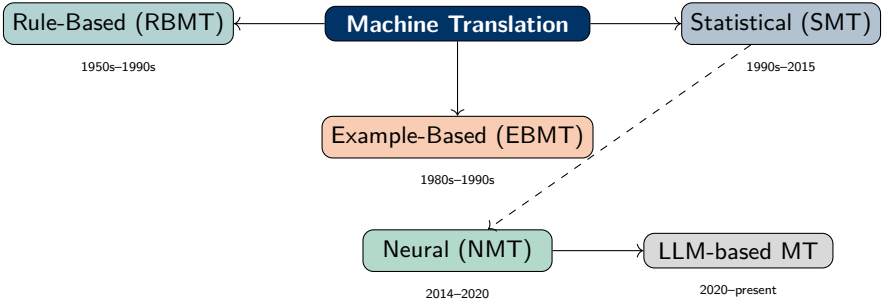
The MT Pyramid: Levels of Transfer



Key Insight

Higher levels = better quality but **more linguistic knowledge** required. Lower levels = faster but **more errors**. Statistical MT operates mostly at the **word/phrase** level.

MT Approaches: An Overview



MT History Timeline

1822	Champollion deciphers Egyptian hieroglyphics using Rosetta Stone
1947	Warren Weaver's memorandum proposes statistical MT
1954	Georgetown-IBM experiment: first public MT demo
1966	ALPAC report halts US MT funding
1970s	Rule-based systems (SYSTRAN, LOGOS) for practical use
1988	IBM proposes statistical/noisy channel approach to MT
1993	Brown et al. publish IBM Models 1–5 (*Computational Linguistics*)
2002	Papineni et al. introduce **BLEU** evaluation metric
2003	Koehn et al. propose **Phrase-Based SMT**
2006	Moses open-source toolkit released (JHU summer workshop)
2007	Chiang proposes **Hierarchical Phrase-Based MT** (Hiero)
2014	Cho et al., Sutskever et al. propose **encoder-decoder NMT**
2015	Bahdanau et al. introduce **attention mechanism** for NMT
2017	Vaswani et al. "**Attention is All You Need**" (Transformer)
2020	**mBART, mT5** and large pretrained multilingual models
2022	Meta AI releases **NLLB-200** (200 languages)

Why is MT Hard? Challenges

Linguistic Challenges:

- **Ambiguity**: "I saw the man with the telescope"
- **Idioms**: "kick the bucket" $\neq$ literal
- **Reordering**: SOV vs. SVO languages
- **Morphology**: agglutinative languages (Turkish, Finnish, Myanmar)
- **Pronoun dropping**: null-subject languages

Computational Challenges:

- **Search problem**: exponential translation space
- **Data sparsity**: low-resource language pairs
- **Domain adaptation**: general $\neq$ medical/legal
- **Evaluation**: no single "correct" translation
- **Context**: coreference across sentences

Myanmar Example —Ambiguity

ခေါင်းထဲမှာ ဘာရှိလဲ။
English: "What is inside the **head**?" or "What is inside the **coffin**?"
(Context disambiguates!)

Modern MT Applications

Practical Applications:

- **Web content** localization (Google Translate)
- **E-commerce** product descriptions
- **Medical** record translation
- **Legal** document translation
- **Subtitle** generation for videos
- **Customer support** chat
- **Social media** content moderation

Special Use Cases in Myanmar:

- Myanmar $\leftrightarrow$ ethnic languages (Shan, Karen, Rakhine, Kachin, Pa'O, Kayah)
- Myanmar $\leftrightarrow$ Myanmar Sign Language
- Myanmar Braille translation
- Burmese dialect translation (Mandalay, Rakhine, Dawei, Tavoy)
- Low-resource MT research

Rule-Based MT (RBMT) —Overview

Definition

Rule-Based MT uses hand-crafted **linguistic rules** (grammar, morphology, syntax) and **bilingual dictionaries** to translate text.

Three main paradigms:

1. **Direct Transfer** —word-for-word replacement using dictionaries
2. **Transfer-Based** —parse source, transfer structure, generate target
3. **Interlingual** —convert source to a language-neutral representation

Well-known RBMT systems:

- **SYSTRAN** (1968, used by the EU and US military)
- **LOGOS** (1970s, commercial)
- **METAL** (Siemens, 1980s)
- **OpenLogos** (open-source version of LOGOS)

Direct Transfer RBMT

Approach: Translate word by word using a bilingual dictionary, with minimal structural analysis.

Steps:

1. Tokenize the source sentence
2. Look up each word in the dictionary
3. Apply simple morphological rules
4. Output the translated words in source order

Example

Source (Myanmar): ကတ်ကို ဖြတ်ပါ
Word lookup:

- ကတ် → card
- ကို → (object marker, skip)
- ဖြတ် → cut
- ပါ → (polite, skip)

Output: "card cut" (incorrect word order!)

Limitation

Direct transfer ignores syntax, so it fails badly for language pairs with different word order (e.g., Myanmar SOV vs. English SVO).

Transfer-Based and Interlingual RBMT

Transfer-Based RBMT:

1. **Analysis**: Parse source sentence (morphology + syntax)
2. **Transfer**: Map source parse tree to target parse tree using bilingual transfer rules
3. **Generation**: Generate target sentence from target parse tree

Better than direct, but transfer rules are **language-pair specific**.

Interlingual RBMT:

1. **Analysis**: Convert source to universal *interlingua* (language-neutral representation)
2. **Generation**: Generate any target language from interlingua

Very powerful in theory; **but constructing a universal interlingua is extremely difficult** in practice.

RBMT Advantages

- Predictable and controllable output
- Good for well-defined domains
- No training data required

RBMT Disadvantages

- Requires expert linguists (expensive)
- Hard to maintain and scale
- Coverage is limited by rules
- Poor handling of ambiguity

RBMT: Why We Need Something Better

- Writing comprehensive rules for even one language pair requires **years of work** by expert linguists
- Rules **interact in complex ways** and are hard to debug
- For n languages, you need $O(n^2)$ rule sets for all pairs
- Real-world text is **messy and irregular**

The Data-Driven Revolution

Key insight (Warren Weaver, 1947; IBM, 1988):

Instead of writing rules, learn from data!

If we have millions of **parallel sentences** (same content in two languages), a machine can **learn** how to translate statistically, without explicit rules.

$\Rightarrow$ This insight led to **Statistical Machine Translation (SMT)**

Example-Based MT (EBMT)

Concept

EBMT was proposed by Makoto Nagao (1984). It translates by **analogy**: find similar source sentences in a *translation memory*, adapt their translations to the new input.

EBMT Steps:

1. **Match**: Find the most similar source sentence(s) in the example base
2. **Adapt**: Modify the matched translation to fit the new source sentence
3. **Combine**: Merge adapted fragments into the final translation

Analogy Example

Known: "私は東京に行く" → "I go to Tokyo"

Input: "私は大阪に行く"

Match: Same except 東京 → 大阪

Output: "I go to Osaka"

Advantages

- Human-quality translations for seen cases
- Reuses validated translations
- Easy to interpret

Limitations

- Fails on unseen sentences
- Brittle combination step
- Hard to scale

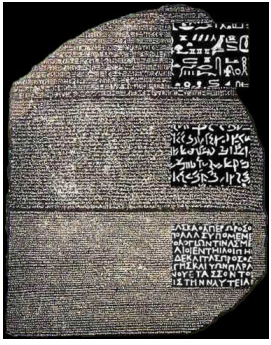
EBMT in Modern MT

- EBMT ideas live on in modern **translation memory (TM)** systems used in professional CAT tools (e.g., SDL Trados, memoQ)
- The concept of **phrase reuse** is directly incorporated in Phrase-Based SMT
- Modern NMT systems sometimes augment with TM for high-precision domains
- **Retrieval-Augmented MT** combines neural MT with example retrieval

Key Idea Carried Forward

EBMT showed that **"learning from examples"** is a valid strategy. Statistical MT formalizes this idea with probability theory.

The Origin of Parallel Corpora: The Rosetta Stone



The Rosetta Stone

Discovered in 1799, it contains the same decree in three scripts: Hieroglyphic, Demotic, and Greek. It is the most famous example of a **Trilingual Parallel Corpus**.

Jean-François Champollion

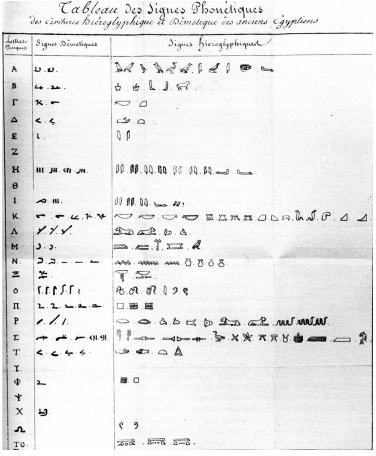
In 1822, Champollion deciphered the Hieroglyphs by **aligning** known Greek names with Egyptian symbols. This was the first "Statistical Alignment" in history.

The Origin of Parallel Corpora: The Rosetta Stone

Connection to SMT

Statistical Machine Translation (SMT) follows this exact principle: by observing patterns across millions of parallel sentences, the computer learns to "decipher" one language using the other, just as Champollion did.

Champollion's "Alignment" Logic



Champollion's Phonetic Alphabet Table

The "Decoding" Process:

1. Champollion focused on **Cartouches**, the oval shapes containing royal names like *Ptolemy* and *Cleopatra*.
2. He assumed that the Greek and Egyptian names occupied the same positions in the parallel text.
3. By comparing different names, he identified that certain symbols always represented the same **phonetic sounds** (e.g., the 'P' and 'L' in both names).
4. This systematic cross-referencing proved that Hieroglyphs were a complex **phonetic system**, not just symbolic pictures.
5. This manual mapping is identical to the **Word Alignment** logic used in IBM Models to find word-to-word links.

Why Statistical Machine Translation?

- **Data is cheap; rules are expensive**
- Billions of parallel sentences exist online (EU proceedings, UN documents, Wikipedia, news archives)
- Statistical models **learn translation patterns automatically**
- Can handle **ambiguity probabilistically**
- Easy to **extend to new language pairs** (just add data)

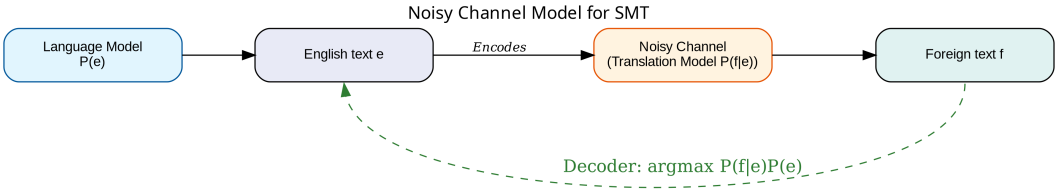
The Fundamental SMT Insight (IBM, 1988–1993)

$$\hat{e} = \arg\max_{e} P(e \mid f)$$

$$= \arg\max_{e} \underbrace{P(f \mid e)}_{\text{Translation Model}} \cdot \underbrace{P(e)}_{\text{Language Model}}$$

Noisy Channel Model: imagine e (English) was "garbled" (encoded) into f (foreign). We want to recover the most likely e.

The Noisy Channel Model —Explained Simply



Three components of SMT:

1. **Translation Model** $P(f \mid e)$: How likely is the foreign sentence given the English? Learned from parallel corpora.
2. **Language Model** $P(e)$: How likely is the English sentence? Learned from monolingual English text.
3. **Decoder**: Search algorithm that finds the best $\hat{e}$ that maximizes the product.

The Noisy Channel Model —Explained Simply

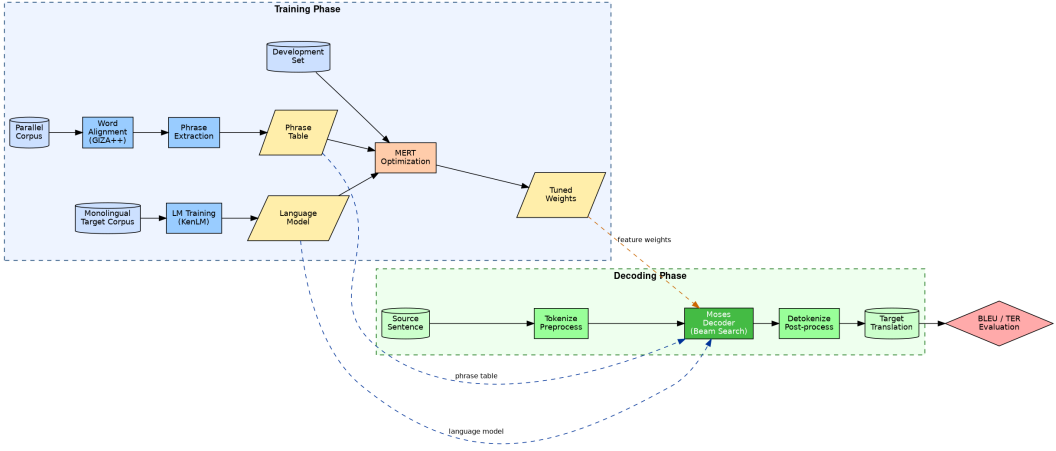
Three components of SMT:

1. **Translation Model** $P(f \mid e)$: probability of source given target. Learned from parallel corpora.
2. **Language Model** $P(e)$: how fluent is the English output? Learned from monolingual text.
3. **Decoder**: finds $\hat{e} = \arg\max_e P(f|e)\, P(e)$.

Why this works

$P(f|e)$ ensures the translation is **faithful** to the source.
$P(e)$ ensures the translation is **fluent** English.

SMT System Architecture



SMT System Architecture

Training Phase:

- Parallel corpus $\rightarrow$ Translation Model
- Monolingual corpus $\rightarrow$ Language Model
- Development set $\rightarrow$ Optimization (MERT)

Decoding Phase:

- Source sentence in
- Decoder searches for best translation
- Target sentence out

BLEU: Automatic MT Evaluation

BLEU Score (Papineni et al., 2002)

Bilingual Evaluation Understudy —the standard automatic metric for MT quality.

Intuition: Compare MT output against human reference translations by counting **n-gram overlaps**.

$$\text{BLEU} = \underbrace{\text{BP}}_{\text{Brevity Penalty}} \times \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{1}$$

- p_n = modified precision for n-grams (1, 2, 3, 4)
- $w_n = 1/N$ (uniform weights, typically $N = 4$)
- BP $= 1$ if output length $\geq$ reference; $e^{1-r/c}$ otherwise
- Score: 0 (worst) to 100 (best); human level $\approx 50+$

Caution

BLEU is not perfect! It does not capture meaning, fluency, or adequacy directly. Always combine with human evaluation for important decisions.

BLEU: Bilingual Evaluation Understudy

- **Method:** Counts overlapping n-grams (1-gram, 2-gram, etc.)

Reference (Human):
ကျွန်တော် ထမင်း စားသည်

MT Output (Machine):
ကျွန်တော် စားသည် ထမင်း

N-gram	Matches	Precision
1-gram	ကျွန်တော်, စားသည်, ထမင်း	$3/3 = 1.0$
2-gram	(none match)	$0/2 = 0.0$

Takeaway

Even if all words are correct (1-gram=1.0), the score drops if the **order** is wrong (2-gram=0.0). This is why BLEU captures "fluency."

Parallel Corpora —The Fuel of SMT

- SMT needs large amounts of **parallel text** (same content in source and target language)
- Well-resourced pairs: English-French, English-German, English-Chinese
- Low-resource: Myanmar ethnic language pairs, Pacific languages

Major Parallel Corpora:

- **Europarl**: EU Parliament proceedings (21 languages)
- **UN Parallel Corpus**: 6 UN languages
- **OPUS**: Open parallel corpus collection
- **CCAligned**: CommonCrawl web data
- **FLORES**: Low-resource benchmark

Myanmar Corpora (Ye Kyaw Thu)

- ALT (Asian Language Treebank): 20k sentences
- myPar: Myanmar ethnic language pairs (preparation in progress)
- MSL4Emergency: Myanmar Sign Language corpus
- myBraille: Myanmar Braille corpus (preparation in progress)
- github.com/ye-kyaw-thu

IBM Models —The Foundation of SMT

Brown et al. (1993) — Computational Linguistics

"The Mathematics of Statistical Machine Translation: Parameter Estimation" — one of the most cited papers in NLP.

- Proposed by Peter Brown, Vincent Della Pietra, Stephen Della Pietra, and Robert Mercer at **IBM Research**
- Introduced five **word alignment models** (IBM Model 1–5)
- All models estimate $P(\mathbf{f} \mid \mathbf{e})$ where:
  - $\mathbf{f} = f_1, f_2, \ldots, f_m$ is the source (foreign) sentence
  - $\mathbf{e} = e_1, e_2, \ldots, e_l$ is the target (English) sentence
- Trained using the **Expectation-Maximization (EM)** algorithm

Important

We typically model $P(f \mid e)$ rather than $P(e \mid f)$ for mathematical convenience. The noisy channel model then uses Bayes' rule.

Word Alignment —The Core Concept

What is Word Alignment?

A word alignment $\mathbf{a}$ maps each source word f_j to a target word $e_{a(j)}$ (or to a special **NULL** word if f_j is untranslated).

Example alignment:

English:	Mary	did not	slap	the	witch
	↓	↓	↓	↓	↓
Spanish:	Maria	no	daba	una	bruja

Myanmar Example

English: I love Myanmar
Myanmar: ကျွန်တော် မြန်မာကို ချစ်သည်
Alignment: $a(1) = 1, a(2) = 3, a(3) = 2$

Note the **reordering**: verb comes last in Myanmar (SOV)!

IBM Model 1 —The Simplest Model

Key assumption: alignment probability is **uniform** —any source word is equally likely to align to any target word.

IBM Model 1 Formula

$$P(\mathbf{f}, \mathbf{a} \mid \mathbf{e}) = \frac{\epsilon}{(l+1)^m} \prod_{j=1}^{m} t(f_j \mid e_{a(j)}) \tag{2}$$

- $t(f \mid e)$ = **lexical translation probability** (how likely is f a translation of e?)
- ϵ = a small constant
- $(l+1)^m$ = normalization over all possible alignments
- l = length of English sentence (target)
- m = length of foreign sentence (source)

IBM Model 1 —The Simplest Model

Summing over all alignments:

$$P(\mathbf{f} \mid \mathbf{e}) = \sum_{\mathbf{a}} P(\mathbf{f}, \mathbf{a} \mid \mathbf{e}) = \frac{\epsilon}{(l+1)^m} \prod_{j=1}^{m} \sum_{i=0}^{l} t(f_j \mid e_i) \tag{3}$$

IBM Model 1 —Understanding the Formula

Let us break down the formula step by step:

- $t(f_j \mid e_i)$: probability that English word e_i translates to foreign word f_j. *Example*: $t(\text{မြန်မာ} \mid \text{Myanmar}) \approx 0.95$
- $\prod_{j=1}^{m}$: multiply probabilities over all m source words (independence assumption!)
- $\sum_{i=0}^{l} t(f_j \mid e_i)$: sum over all possible target words that f_j could align to (including NULL $= e_0$)
- $\frac{\epsilon}{(l+1)^m}$: uniform prior over alignments

Intuition

Model 1 says: generate each source word **independently** by picking a random target word and translating it. No reordering model, no fertility. Very simple but surprisingly effective as a **seed** for richer models.

The EM Algorithm for IBM Model 1

Problem: We observe $(\mathbf{f}, \mathbf{e})$ pairs but not the alignments $\mathbf{a}$ (hidden variables). Use **EM**!

EM Algorithm —Two Steps

E-step (Expectation) Compute **expected counts** of how many times word e_i aligns to f_j using current parameters t:

$$c(f \mid e) \mathrel{+}= \frac{t(f \mid e)}{\sum_{e'} t(f \mid e')} \quad \text{(soft count)} \tag{4}$$

M-step (Maximization) **Re-estimate** parameters to maximize expected log-likelihood:

$$t(f \mid e) = \frac{c(f \mid e)}{\sum_{f'} c(f' \mid e)} \tag{5}$$

Repeat E- and M-steps until convergence (typically 5–10 iterations). Model 1 is **convex** — guaranteed to reach the global optimum!

EM Algorithm —Small Example

Suppose we have two parallel sentences:

1. "the house" ↔ "အိမ်"
2. "the dog" ↔ "ခွေး"

Initialization: All $t(f \mid e) = 0.5$ (uniform)

Iteration 1 E-step: Compute soft counts:

Foreign	English	Soft count
အိမ်	the	0.5
အိမ်	house	0.5
ခွေး	the	0.5
ခွေး	dog	0.5

Iteration 1 M-step: Normalize ⇒ still uniform

After many iterations: $t(\text{အိမ်} \mid \text{house}) \approx 1$, $t(\text{ခွေး} \mid \text{dog}) \approx 1$
⇒ the article *the* is split between both foreign words equally.

IBM Model 2 —Adding Position

Key improvement over Model 1: Add an **alignment probability** $a(i \mid j, m, l)$ that captures position preference.

IBM Model 2 Formula

$$P(\mathbf{f}, \mathbf{a} \mid \mathbf{e}) = \prod_{j=1}^{m} t(f_j \mid e_{a(j)}) \cdot a(a(j) \mid j, m, l) \tag{6}$$

- $a(i \mid j, m, l)$ = probability that source position j aligns to target position i, given sentence lengths m and l
- This captures the intuition that words **close in position** are more likely to align

EM Training

E-step: compute soft alignment counts weighted by both t and a
M-step: re-estimate $t(f \mid e)$ and $a(i \mid j, m, l)$

IBM Model 3 —Adding Fertility

New concept: **Fertility** ϕ_i = number of foreign words that English word e_i generates (can be 0, 1, 2, ...).

IBM Model 3 Formula

$$P(\mathbf{f}, \mathbf{a} \mid \mathbf{e}) = \prod_{i=1}^{l} \underbrace{n(\phi_i \mid e_i)}_{\text{fertility}} \cdot \phi_i! \cdot \prod_{j:a(j)=i} t(f_j \mid e_i) \cdot \prod_{j=1}^{m} d(j \mid a(j), m, l) \tag{7}$$

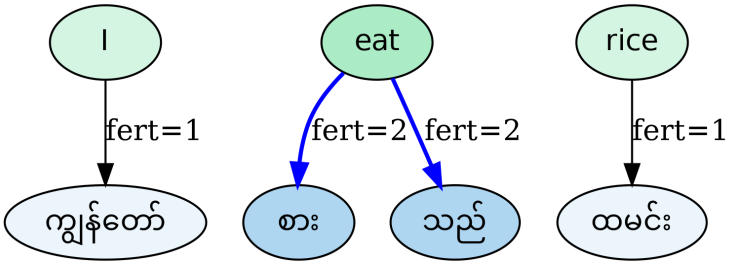
Three new parameters:

- $n(\phi \mid e)$: probability that word e has fertility ϕ
- $d(j \mid i, m, l)$: **distortion** probability (where source word lands)
- p_0, p_1: probabilities for inserting NULL-generated words

Example

English "the" may have fertility 0 when translating to Myanmar (no article in Myanmar language).
English "not" may generate two words in some languages (double negation). e.g., ဖြစ် x ဘူး

Fertility of IBM Model 3



How words have children

Knowing “eat” might become “စား” + “သည်”.

IBM Model 4 —Better Reordering

Key improvement: Model 4 uses a more sophisticated **distortion model** based on **word classes** (POS tags or clusters).

IBM Model 4 Distortion

Instead of absolute position $d(j \mid i, m, l)$, Model 4 uses:

- $d_1(\Delta j \mid A(e_i), B(f_j))$: probability of **head displacement** $\Delta j = j - \text{center}(cept(i))$, conditioned on word classes
- $d_{>1}(\Delta j \mid B(f_j))$: probability of non-head displacement
- $A(\cdot), B(\cdot)$: word class functions (e.g., POS tags)

IBM Model 5 —Vacancy-based Distortion

IBM Model 5: Further refines vacancy-based distortion (avoids deficiency).

Summary: IBM Models

Model	Lex. t	Alignment	Fertility	Distortion
1	✓	Uniform	—	—
2	✓	Position	—	—
3	✓	—	✓	Absolute
4	✓	—	✓	Class-based
5	✓	—	✓	Vacancy

The IBM models are incrementally built, with each version introducing a new linguistic constraint to address the limitations of its predecessor. While early models (1–2) ignore word count and movement, later models (3–5) introduce fertility (how many words) and sophisticated distortion (where they go) to handle complex reordering. This progression shifts from a simple uniform alignment toward a vacancy-based system that ensures a more mathematically sound and grammatically correct translation.

Word Alignment in Practice: GIZA++

GIZA++ (Och & Ney, 2003)

Open-source implementation of IBM Models 1–4.

- Standard tool for word alignment in SMT
- Trains models sequentially: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
- Each model initializes from the previous one
- Outputs Viterbi alignments (best alignment per sentence)

Bidirectional Alignment

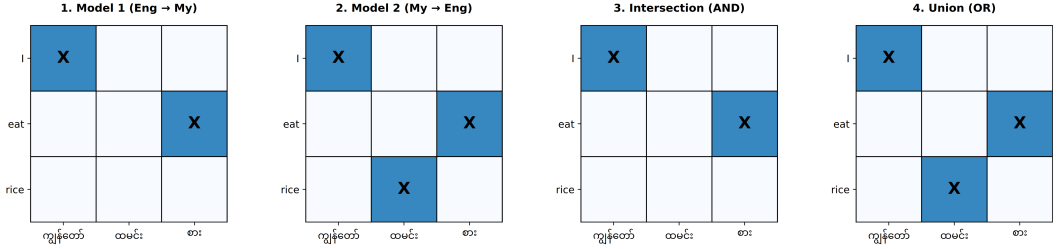
Run GIZA++ in both directions:
$\mathbf{e} \rightarrow \mathbf{f}$ and $\mathbf{f} \rightarrow \mathbf{e}$

Then **symmetrize** alignments:
grow-diag-final-and
(default in Moses)

Other tools:

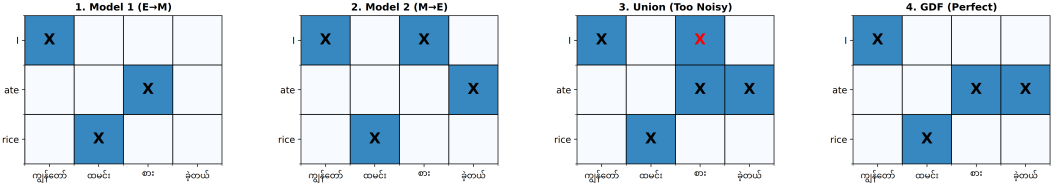
- **fast_align**: log-linear reparameterization
- **eflomal**: Bayesian alignment
- **pialign**: phrasal ITG aligner
- **anymalign**: multilingual subcorpora sampling
- **awesome-align**: BERT-based

SMT Word Alignment Symmetrization



- In SMT, directional models (IBM Models) are "many-to-one." English → Myanmar might fail to find a word because of a complex phrase structure.
- * The **Intersection** is very "safe" (high precision) but usually leaves many words unaligned.
- * The **Union** is very "greedy" (high recall) and ensures more words are covered, which is essential for building a complete Phrase Table.

Grow-Diag-Final



- **Intersection:** Misses the verb.
- **Union:** The Union is “too greedy”—it accepts every mistake from both models.
- **Grow-Diag-Final (GDF):** It grows to catch the verb but ignores the noise.

Word Alignment Example

GIZA++ alignment output format:

```
# Input sentence pair
English: I love Myanmar
Myanmar: kyandaw myamar ko chittae

# Viterbi alignment (source-index target-index)
0-0 1-3 2-1
```

This means:

- "I" aligns to word 0 (ကျွန်တော်)
- "love" aligns to word 3 (ချစ်တယ်)
- "Myanmar" aligns to word 1 (မြန်မာ)

Running GIZA++

```
# Train Model 1 to 4
plain2snt.out eng.txt my.txt
snt2cooc.out eng.vcb my.vcb \
  eng-my.snt > eng-my.cooc
GIZA++ -S my.vcb -T eng.vcb \
  -C my-eng.snt \
  -o output_prefix
```

Note

In modern Moses, word alignment is **automated** in the training pipeline. You rarely call GIZA++ manually.

Phrase–Based SMT: Motivation

- **Problem with word-based models**: many-to-many translations are common (idioms, collocations, multi-word expressions)
- **Solution**: translate **phrases** (sequences of words) instead of single words
- Key paper: **Koehn, Och, Marcu (2003)** — NAACL 2003

Example: Phrase Translation

Source (German): "Morgen fliege ich nach Kanada zur Konferenz"

Phrase segmentation:
[Morgen] [fliege ich] [nach Kanada] [zur Konferenz]

Phrase translation:
[Tomorrow] [I will fly] [to Canada] [for the conference]

English output: "Tomorrow I will fly to Canada for the conference" [Koehn 2008]

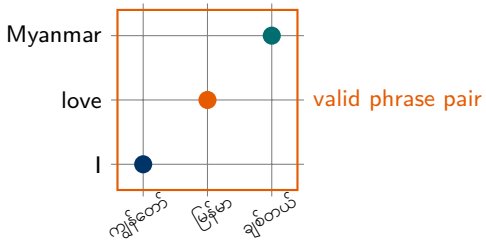
Phrase Extraction Algorithm

Step 1: Obtain word alignments (GIZA++ bidirectional)
Step 2: Extract all **consistent phrase pairs** from alignments

Consistency Condition

A phrase pair $(\bar{f}, \bar{e})$ is **consistent** with alignment **a** if:

1. Every foreign word in $\bar{f}$ that is aligned maps to a word in $\bar{e}$
2. Every target word in $\bar{e}$ that is aligned maps to a word in $\bar{f}$
3. At least one word in $\bar{f}$ is aligned



Phrase Translation Table

After extracting all phrase pairs, estimate phrase translation probabilities:

Phrase Probability Estimation

$$\phi(\bar{f} \mid \bar{e}) = \frac{\text{count}(\bar{f}, \bar{e})}{\text{count}(\bar{e})} \tag{8}$$

$$\phi(\bar{e} \mid \bar{f}) = \frac{\text{count}(\bar{f}, \bar{e})}{\text{count}(\bar{f})} \tag{9}$$

Both directions are used as features in the log-linear model.

Additional scores stored in the phrase table:

- $\phi(\bar{f} \mid \bar{e})$ and $\phi(\bar{e} \mid \bar{f})$: phrase translation probabilities
- $\text{lex}(\bar{f} \mid \bar{e})$ and $\text{lex}(\bar{e} \mid \bar{f})$: lexical weighting
- Phrase penalty (constant, penalizes using many short phrases)

Example Phrase Table Entry

ချစ်သည် ||| love ||| 0.9 0.85 0.95 0.9 ||| 0-0 ||| 100

The Log-Linear Model for Phrase-Based SMT

Log-Linear Combination (Och & Ney, 2002)

$$P(\mathbf{e} \mid \mathbf{f}) \propto \exp\left(\sum_{i=1}^{n} \lambda_i \cdot h_i(\mathbf{e}, \mathbf{f})\right) \tag{10}$$

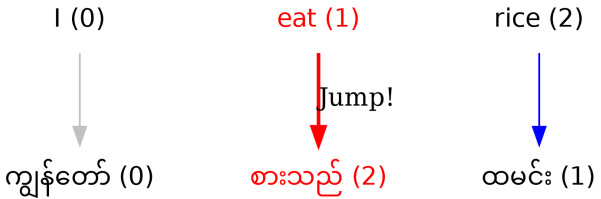
The best translation is:

$$\hat{\mathbf{e}} = \arg\max_{\mathbf{e}} \sum_{i=1}^{n} \lambda_i \cdot h_i(\mathbf{e}, \mathbf{f}) \tag{11}$$

Standard feature functions h_i:

1. h_1: Phrase translation probability $\log \phi(\bar{f} \mid \bar{e})$
2. h_2: Inverse phrase probability $\log \phi(\bar{e} \mid \bar{f})$
3. h_3: Lexical weighting $\log \text{lex}(\bar{f} \mid \bar{e})$
4. h_4: Inverse lexical weighting $\log \text{lex}(\bar{e} \mid \bar{f})$
5. h_5: Language model $\log P_{LM}(\mathbf{e})$
6. h_6: Word penalty
7. h_7: Phrase penalty
8. h_8: Reordering model

Reordering



Some words are “jumpy” by nature. For example, in Myanmar, verbs almost always jump to the end of the sentence. A lexical reordering model learns this behavior for specific words.

Reordering Models

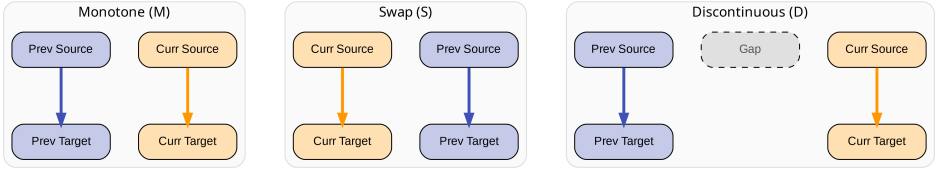
- Source and target languages often have different word orders
- **Monotonic** translation: phrases in same order as source
- **Reordering**: some phrases are swapped

Distance-Based Reordering (Simple)

Penalize **large jumps** in phrase alignment:

$$d(\mathbf{e}, \mathbf{f}) = \exp\left(-\lambda \sum_{\text{phrase}} |e_{\text{start}} - f_{\text{prev\_end}}|\right) \tag{12}$$

Reordering Models



Lexicalized Reordering (Galley & Manning, 2008)

For each phrase pair, estimate probabilities of:

- **Monotone (M)**: phrase follows the previous phrase in order
- **Swap (S)**: phrase and previous phrase are swapped
- **Discontinuous (D)**: non-adjacent reordering

$p(\mathrm{M/S/D} \mid \bar{e}, \bar{f})$ stored in reordering table.

Practical Example: Distortion Penalty (Reordering)

- **Concept:** How far does the "cursor" jump to find the next word?
- **Formula:** $d_i = |\text{start}_i - \text{end}_{i-1} - 1|$

Python Implementation: English (SVO) → Myanmar (SOV)

```python
def calculate_distortion(prev_end, curr_start):
    # |current_start - prev_end - 1|
    return abs(curr_start - prev_end - 1)

# English: I(0) eat(1) rice(2) -> Myanmar: I(0) rice(1) eat(2)
# 1. Translate "I" (Source Pos 0)
d1 = calculate_distortion(-1, 0) # Score: 0

# 2. Translate "rice" (Source Pos 2)
d2 = calculate_distortion(0, 2)  # Score: 1 (Jump!)

# 3. Translate "eat" (Source Pos 1)
d3 = calculate_distortion(2, 1)  # Score: 2 (Jump back!)

total_penalty = d1 + d2 + d3 # Total: 3
```

Intuition for Students

In SMT, we want to **minimize** this penalty. A very high score suggests the translation might be ungrammatical or the alignment is too chaotic.

Phrase-Based SMT: Myanmar Example

Phrase-based translation: Myanmar → English

Source (မြန်မာ): ကျွန်တော် မနက်ဖြန် ရန်ကုန်မြို့ကို သွားမည်

Phrase segmentation:

1. ကျွန်တော် → "I"
2. မနက်ဖြန် → "tomorrow"
3. ရန်ကုန်မြို့ကို → "to Yangon"
4. သွားမည် → "will go"

With reordering: "I will go to Yangon tomorrow"
(Verb moved to match English SVO order, time adverb repositioned)

Key Advantage

Phrase-based SMT handles **local reordering** and **multi-word expressions** much better than word-based models.

Factored Translation Models

Koehn & Hoang (2007) — Factored Translation Models

Extend phrase-based MT by using **multiple linguistic factors** as source/target tokens.

Factors can include:

- Surface word form
- POS tag
- Lemma / stem
- Morphological analysis
- Syntactic chunk label

Source token: `word|POS|morph`
Target token: `word|POS|morph`

Factored Myanmar Example

ဆောင်ရွက်ပါ|VERB|IMP

Factor 1: surface = ဆောင်ရွက်ပါ
Factor 2: POS = VERB
Factor 3: morph = Imperative

Useful for morphologically **rich** languages like Myanmar, Turkish

Phrase-Based SMT: Summary & Limitations

Strengths

- Handles multi-word expressions
- Local context through phrases
- Strong empirical performance (2003–2014)
- Interpretable (phrase table visible)
- Moses: easy to train and use

Limitations

- Limited **long-range reordering**
- Phrase table can be huge
- Independence between phrases
- Poor morphology handling
- No global syntactic structure

$\Rightarrow$ These limitations motivated **Hierarchical Phrase-Based SMT** and eventually **Syntax-Based SMT**.

Hierarchical Phrase-Based SMT (Hiero)

Chiang (2005, 2007) — "A Hierarchical Phrase-Based Model for SMT"

Key idea: extend phrase-based MT with **non-terminal symbols** that allow recursive phrase substitution, capturing long-range reordering.

Uses Synchronous Context-Free Grammars (SCFG):

$$X \rightarrow \langle \gamma, \alpha \rangle \tag{13}$$

where γ is a source string with X non-terminals, and α is the corresponding target string.

Example SCFG Rules

- $X \rightarrow \langle$ ဂါ, I $\rangle$
- $X \rightarrow \langle X_1$ နှင့် X_2, X_1 and $X_2 \rangle$
- $X \rightarrow \langle X_1$ တို့ X_2, X_2 $X_1 \rangle$ (allows reordering!)

Synchronous CFG and CKY Decoding

SCFG rules are extracted from:

1. Phrase pairs (same as phrase-based)
2. Hierarchical rules with **gaps** (non-terminals X_1, X_2)

Decoding: Uses the **CKY algorithm** (bottom-up chart parsing)

- Fill a parse chart over source sentence
- Each cell stores the best translation hypothesis
- Beam search prunes the search space

Hiero Advantages

- Handles **long-range** reordering
- No syntactic annotation needed
- Purely data-driven
- Grammar is learned from parallel text

Limitation

Very large rule tables.
CKY decoding is slower than stack-based decoding.

Syntax-Based SMT Variants

Beyond Hiero, **linguistically-informed** syntax models:

String-to-Tree (Galley et al., 2006):

- Source: flat string
- Target: parsed tree structure
- Uses target-language parser

Tree-to-String (Liu et al., 2006):

- Source: parsed tree
- Target: flat string
- Uses source-language parser

Tree-to-Tree (Yamada & Knight, 2001):

- Both sides parsed
- Highest quality but requires two parsers
- Very expensive

Moses Support

Moses includes both phrase-based and tree-based (Hiero) decoding. Select with `-search-algorithm 1` for CKY.

Hiero vs. Phrase-Based: Key Differences

Aspect	Phrase-Based	Hiero
Translation units	Flat phrases	Hierarchical rules
Reordering	Local (distortion limit)	Long-range (via grammar)
Grammar	None	SCFG
Decoding	Stack-based	CKY chart parsing
Syntax	No	Implicit
Speed	Faster	Slower
Rule count	Millions of phrases	Even more rules
Performance	Good	Better for distant pairs

When to Use Hiero?

Hiero tends to outperform phrase-based MT for language pairs with **very different word orders**, such as Chinese-English, Japanese-English, or **Myanmar-English** (SOV↔SVO).

Operation Sequence Model (OSM)

Durrani et al. (2011) — "A Joint Sequence Translation Model with Integrated Reordering"

OSM models translation as a **sequence of operations** that simultaneously generate source and target words, integrating translation and reordering into a single model.

Five operations defined:

1. **Generate(s, t)**: produce source word s and target word t simultaneously
2. **Insert Gaps (k)**: insert k gaps (placeholder for reordering)
3. **Jump Back (k)**: move k positions back (to fill a gap)
4. **Jump Forward**: advance to the next unfinished position
5. **Generate Only Source (s)**: generate s with no target word (deletion)

OSM: Formula and Intuition

OSM Probability

$$P(\mathbf{o}_1^n) = \prod_{k=1}^{n} P(o_k \mid o_{k-N+1}, \ldots, o_{k-1}) \tag{14}$$

$\mathbf{o}_1^n$ is the sequence of operations; modeled as an **N-gram language model over operations**.

Key advantages over phrase-based:

- Reordering and translation are **jointly modeled**
- No need for separate reordering model
- Captures **long-distance** dependencies through operation sequences
- Can be added as a **feature** in Moses alongside phrase-based

Research Result (Ye Kyaw Thu et al., 2021)

Compared Phrase-based, Hierarchical, and OSM for Burmese–Pa'O language pair. All three models were combined (system combination) for best results.
Reference: *"SMT System Combinations on Phrase-based, Hierarchical Phrase-based and OSM for Burmese and Pa'O Language Pair"*, JIIST 2021.

OSM: Myanmar Application

OSM for Myanmar–Pa'O Translation:

- Pa'O is a Tibeto-Burman language spoken in Shan State, Myanmar
- SOV word order (similar to Myanmar)
- Small parallel corpus (low-resource setting)
- OSM captures operation patterns unique to this pair

System combination (Moses):

- Phrase-based + Hierarchical + OSM combined using **MERT**
- Combined system outperforms individual models

OSM vs. Phrase-based (BLEU)

System	bm-po	po-bm
Phrase-based	33.04	41.20
Hierarchical	33.23	41.70
OSM	35.06	43.45
Combined 3	**38.45**	**47.57**

OSM: Summary

OSM Strengths

- Unified model for translation + reordering
- Better long-range movement
- Works well for distant language pairs
- Can be used alongside other models

OSM Limitations

- Training is more complex
- Larger n-gram operation model
- Less interpretable than phrase table
- Still not as good as NMT for fluency

Usage in Moses

OSM is integrated as a feature in Moses. Enable with:
`--operation-sequence-model yes`
Requires SRILM or KenLM for the operation language model.

Language Models in SMT

Role of the Language Model

The Language Model (LM) scores how **fluent** and **grammatical** the target language output is, independently of the source sentence.

$$P(\mathbf{e}) = P(e_1, e_2, \ldots, e_l) \tag{15}$$

A good LM assigns high probability to grammatical English and low probability to word salad.

- Trained on **large monolingual** target-language corpora
- For English: billions of words (Common Crawl, Wikipedia, news)
- For Myanmar: smaller corpora, but growing
- The LM is the "spell-checker" of MT —it ensures fluent output

N-gram Language Model

Assumption (Markov): each word depends only on the previous $n-1$ words.

N-gram LM Formula

$$P(e_1, e_2, \ldots, e_I) = \prod_{i=1}^{I} P(e_i \mid e_{i-n+1}, \ldots, e_{i-1}) \tag{16}$$

Using the **chain rule** and the n-gram approximation:

$$P(e_i \mid e_1, \ldots, e_{i-1}) \approx P(e_i \mid e_{i-n+1}, \ldots, e_{i-1}) \tag{17}$$

Common choices of n:

- **Unigram** ($n = 1$): $P(e_i)$ — bag of words
- **Bigram** ($n = 2$): $P(e_i \mid e_{i-1})$
- **Trigram** ($n = 3$): $P(e_i \mid e_{i-1}, e_{i-2})$
- **5-gram**: standard in SMT practice

Bigram Example

P(I love Myanmar)
$= P(\text{I}) \times P(\text{love}|\text{I}) \times P(\text{Myanmar}|\text{love})$

High probability: natural English
Low probability: P(Myanmar love I) (ungrammatical)

N-gram Calculation: Step-by-Step

Training Corpus:

1. ကျွန်တော် ထမင်း စားသည်။
2. ကျွန်တော် ရေ သောက်သည်။
3. မောင်မောင် ထမင်း စားသည်။

Bigram Probability: $P(\text{ထမင်း}|\text{ကျွန်တော်})$

Formula: $\frac{Count(\text{ကျွန်တော်},\text{ထမင်း})}{Count(\text{ကျွန်တော်})}$

Calculation: $\frac{1}{2} = 0.5$

Trigram Probability: $P(\text{စားသည်}|\text{ကျွန်တော်}, \text{ထမင်း})$

Formula: $\frac{Count(\text{ကျွန်တော်},\text{ထမင်း},\text{စားသည်})}{Count(\text{ကျွန်တော်},\text{ထမင်း})}$

Calculation: $\frac{1}{1} = 1.0$

The Sparsity Problem & Back-off

- **Problem:** What if we see "ကျွန်တော် ရေ စားသည်"?
- Our data has 0 counts for this trigram → Probability becomes **0**.
- A zero probability in any part of the sentence makes the **whole sentence** probability 0.

Simple Back-off Logic

If $Count(w_1, w_2, w_3) == 0$:

1. "Back-off" to Bigram: $P(w_3|w_2)$
2. Apply a penalty weight (α): $\alpha \times P(w_3|w_2)$

Intuition: If we don't know the full phrase, we guess based on the last two words.

Smoothing: Handling Unseen N-grams

Problem: Many n-grams never appear in training data, so $P(e_i \mid e_{i-1}, \ldots) = 0$, making the whole sentence probability 0. **Solution**: **Smoothing** —redistribute probability mass to unseen events.

Major Smoothing Techniques

Laplace (add-one) Add 1 to all counts:

$$P_{\text{Lap}}(e_i \mid e_{i-1}) = \frac{c(e_{i-1}, e_i)+1}{c(e_{i-1})+V}$$

Jelinek-Mercer Linearly interpolate different n-gram orders:

$$P_{\text{JM}} = \lambda_n P_n + (1 - \lambda_n) P_{n-1}$$

Kneser-Ney (KN) Use **continuation probability** for backoff:
Best-performing in practice; standard for SMT

Modified KN Chen & Goodman (1998) —even better

Kneser-Ney Smoothing —Explained

Key idea: Instead of the raw count of a word, use the **number of different contexts** it appears in.

Kneser-Ney Formula

$$P_{\text{KN}}(w \mid h) = \frac{\max(c(h, w) - d, 0)}{c(h)} + \lambda(h) \cdot P_{\text{KN}}(w) \tag{18}$$

$$P_{\text{KN}}(w) = \frac{|\{v : c(v, w) > 0\}|}{|\{(u, v) : c(u, v) > 0\}|} \tag{19}$$

- d: discount constant (typically 0.75)
- $\lambda(h)$: backoff weight to ensure probabilities sum to 1
- $P_{\text{KN}}(w)$: **continuation probability** (how many unique contexts does w follow?)

Kneser-Ney Smoothing: Beyond Simple Counting

- Standard smoothing only looks at **Frequency**.
- Kneser-Ney looks at **Continuation Probability** (Versatility).

High Frequency

"သည်" (Sentence closer)
Appears often, but only in ONE context (at the end).

High Versatility

"စားသည်" (to eat)
Appears with "ထမင်း", "မုန့်", "အသီး". More likely to follow a new word.

The Logic: If we encounter a new word, it is more likely to be followed by a word that appears in **many different contexts** rather than just a word that is common in one spot.

Kneser-Ney Smoothing —Intuition

Example 1: San Francisco

"Francisco" appears frequently in corpora, but almost always follows "San". Its **continuation probability** is low because it is not a "versatile" word.

Example 2: Myanmar Context (မြန်မာဘာသာဖြင့် ဥပမာ)

"ဂိတ်" (Station) ဟူသောစကားလုံးသည် စာသားထဲတွင် အကြိမ်ရေများစွာပါသော်လည်း များသောအားဖြင့် "မီးရထား" သို့မဟုတ် "ကား" နောက်တွင်သာ တွဲလျက်ပါလေ့ရှိသည်။

- **High Freq, Low Continuation:** "ဂိတ်" သည် Context အနည်းငယ်တွင်သာ ပေါ်တတ်သဖြင့် Backoff ပေးရာတွင် ဦးစားမပေးသင့်ပါ။
- **Low Freq, High Continuation:** "နား" (Ear/Near) ကဲ့သို့ စကားလုံးမျိုးသည် Context ပေါင်းစုံတွင် ပါဝင်နိုင်သဖြင့် Continuation probability ပိုမြင့်ပါသည်။

Key Takeaway (မှတ်ချက်)

စကားလုံးတစ်လုံး၏ ပေါ်ထွက်နှုန်း (Raw Count) တစ်ခုတည်းကို မကြည့်ဘဲ၊ ၎င်းသည် ရှေ့ဆက်စကားလုံး **ဘယ်နှစ်မျိုး** နှင့် တွဲဖက်နိုင်သနည်း (Diversity of contexts) ကို ကြည့်ခြင်းဖြစ်သည်။

LM Training Tools

SRILM (Stolcke, 2002)

- Standard toolkit for n-gram LM training
- Supports Kneser-Ney, Witten-Bell, etc.
- Integrates with Moses

Training command:

```
ngram-count -order 5 \
  -text english.txt \
  -kndiscount \
  -interpolate \
  -lm english.lm
```

KenLM (Heafield, 2011)

- **Much faster** than SRILM
- Memory-efficient (binary format)
- Supports up to 6-gram
- Default in Moses

Training command:

```
lmplz -o 5 \
  < english.txt \
  > english.arpa
build_binary english.arpa \
  english.blm
```

LM Perplexity

Perplexity — Measuring LM Quality

$$\text{PP}(W) = P(w_1, w_2, \ldots, w_N)^{-1/N} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log P(w_i \mid w_{i-n+1}^{i-1})\right) \tag{20}$$

Intuition: Perplexity is the **branching factor** —how many choices the model is "confused" between at each step.

- **Lower perplexity** = better LM (more predictable text)
- Character-level: PP $\approx$ 2–4 (few choices per character)
- Word-level English: PP $\approx$ 50–200 (many word choices)
- Perfect LM: PP = 1 (always predicts correctly)

Evaluation

Always evaluate LM on a **held-out test set** (not training data!). Report perplexity on the same domain as your MT test set.

Perplexity (PP): Measuring "Confusion"

- **Concept:** How well does the probability model predict the sample?
- **Simple Math:** $PP(W) = P(w_1, w_2, ..., w_N)^{-\frac{1}{N}}$

Example: Prediction Probability

Sentence: "ကျွန်တော် ထမင်း စားသည်" (N=3)
Suppose our model gives these probabilities:

- $P(\text{ကျွန်တော်}) = 0.5$
- $P(\text{ထမင်း}|\text{ကျွန်တော်}) = 0.1$
- $P(\text{စားသည်}|\text{ထမင်း}) = 0.2$

Total Probability $= 0.5 \times 0.1 \times 0.2 = \mathbf{0.01}$

Calculation

$PP = (0.01)^{-\frac{1}{3}} \approx \mathbf{4.64}$
Interpretation: On average, the model was choosing between 4.6 words.

LM Integration in SMT —Practical Tips

- Use a **5-gram** LM for SMT (good balance of coverage and sparsity)
- Train on **in-domain** data for best results; use mix of in-domain and general
- **Domain matters**: a medical LM works poorly for chat text
- For low-resource Myanmar languages:
  - Collect monolingual text from news, Wikipedia, social media
  - Augment with **back-translated** data
  - Consider **character or syllable** level LMs for agglutinative languages
- **Multiple LMs** can be used as separate features in the log-linear model

Myanmar LM Resources

- **Wiki**: Myanmar Wikipedia (monolingual)
- **myNews**: Myanmar news corpora
- **OSCAR**: Web-crawled Myanmar text
- `https://github.com/ye-kyaw-thu` for specific domain corpora

Why Optimize Feature Weights?

- The log-linear model combines many features: phrase probabilities, LM, reordering, word penalty ...
- Each feature has a **weight** λ_i that determines its importance
- **Bad weights** $\Rightarrow$ bad translations (even with good features)
- We want to **automatically find** the best weights

Optimization Goal

Find weights $\boldsymbol{\lambda} = (\lambda_1, \ldots, \lambda_n)$ that maximize translation quality on a **development set**:

$$\hat{\boldsymbol{\lambda}} = \arg\max_{\boldsymbol{\lambda}} \text{BLEU}(\boldsymbol{\lambda}) \tag{21}$$

Problem: BLEU is **not differentiable** — can't use gradient descent directly.

MERT: Minimum Error Rate Training

Och (2003) — "Minimum Error Rate Training in SMT"

MERT is the standard optimization algorithm for SMT. It directly optimizes BLEU on the development set by line search.

MERT Algorithm:

1. Start with initial weights $\boldsymbol{\lambda}^{(0)}$
2. **Generate k-best lists**: run decoder with current weights, collect $k = 100$ best translations for each dev sentence
3. For each feature λ_i in turn:
   a. Fix all other weights; vary λ_i over a 1D grid
   b. Find the value of λ_i that maximizes BLEU on dev set
4. Update weights; repeat until convergence

Note

MERT works well for ≤ 20 features but becomes unstable for larger feature sets. Use **PRO** or **MIRA** for more features.

MERT: Finding the Best Weights

The Goal: Maximize BLEU by adjusting weights (λ).

$$\text{Score} = \lambda_1(\text{Translation}) + \lambda_2(\text{Language Model}) + \lambda_3(\text{Distortion})$$

- **Step 1:** Start with random weights (e.g., all 0.5).
- **Step 2:** Translate a small "Tune" set and calculate BLEU.
- **Step 3:** Change weights slightly. Did BLEU go up?
- **Step 4:** Repeat until the "Error Rate" is at its minimum.

Why do we need this?

If we give too much weight to the **Language Model**, the output sounds natural but loses the original meaning. MERT finds the "Sweet Spot." The point where meaning, fluency, and order are perfectly balanced.

PRO and MIRA: Modern Optimization

PRO (Hopkins & May, 2011)

Pairwise Ranking Optimization:

- Sample pairs of translations from k-best list
- Label them (better/worse) based on BLEU
- Train a classifier (logistic regression) to rank correctly
- Works with **hundreds of features**

MIRA (Chiang et al., 2008)

Margin Infused Relaxed Algorithm:

- Online perceptron-style algorithm
- Update weights when a better translation is ranked lower
- Very fast convergence
- Good for **sparse feature sets** (millions of features)

Running MERT in Moses

```
perl mert-moses.pl devset.src devset.ref \
  moses /path/to/moses.ini \
  --decoder-flags "-threads 4" \
  --mertdir /path/to/mert
```

Training Pipeline: Complete SMT Workflow

1. **Data preparation**: tokenize, lowercase, clean parallel corpus
2. **Word alignment**: run GIZA++ in both directions (e→f, f→e)
3. **Symmetrize** alignments (grow-diag-final-and)
4. **Extract phrases** and compute phrase probabilities
5. **Train language model** (KenLM 5-gram) on target monolingual data
6. **Build Moses configuration** file (moses.ini)
7. **Optimize weights** with MERT on development set
8. **Evaluate** on test set (BLEU, TER, METEOR)

Moses train-model.perl

```
perl train-model.perl -root-dir /path/to/model \
  -corpus /path/to/parallel \
  -f my -e en \
  -alignment grow-diag-final-and \
  -reordering msd-bidirectional-fe \
  -lm 0:5:/path/to/lm.blm:8 \
  -external-bin-dir /tools/bin
```

The Decoding Problem

Decoding

Given a source sentence $\mathbf{f}$ and a trained model, find the translation $\hat{\mathbf{e}}$ that maximizes the model score:

$$\hat{\mathbf{e}} = \arg\max_{\mathbf{e}} \sum_i \lambda_i h_i(\mathbf{e}, \mathbf{f}) \tag{22}$$

This is an **NP-hard optimization problem** (combinatorial explosion of candidates).

Why is it hard?

- Source sentence of length m can be segmented in 2^{m-1} ways
- Each segment can be translated by many phrase pairs
- Phrases can be reordered in many ways
- LM depends on full output sentence
- $\Rightarrow$ Exact search is intractable; use **heuristic search**

Stack-Based Decoding (Pharaoh / Moses)

Idea: Build translation **left-to-right**, maintaining a set of partial hypotheses organized into **stacks** by coverage.

Stack Decoding Algorithm

1. Initialize stack 0 with empty hypothesis (no words covered)
2. For each stack n (covering n source words):
   a. Pop the best hypothesis from stack n
   b. Extend it by translating an uncovered source phrase
   c. Add extended hypothesis to stack $n + |\text{phrase}|$
3. Prune each stack to **beam size** b (default: 200)
4. Return the best hypothesis from stack m (all words covered)

Beam Search in SMT Decoding

Beam search keeps only the top-b hypotheses at each step:

- **Beam size** b: trade-off between quality and speed
- $b = 1$: greedy search (fast but often bad)
- $b = 200$: standard Moses default
- Larger b = better quality but slower

Hypothesis score:

$$\text{score}(h) = \sum_i \lambda_i h_i + \text{future\_cost}(h) \qquad (23)$$

Future cost: **heuristic estimate** of completing the translation.

Pruning Strategies

- **Beam pruning**: keep top-b by score
- **Histogram pruning**: hard limit per stack
- **Threshold pruning**: drop hypotheses within α of best
- **Recombination**: merge identical coverage+end states

Hypothesis Recombination

Key optimization: Two hypotheses can be **merged** if they are **equivalent** for future decisions:

Recombination Condition

Two hypotheses h_1 and h_2 are equivalent (and the weaker is discarded) if they have:

1. The **same set of covered source words**
2. The **same last** $n-1$ **target words** (same LM state)

Keep only the higher-scoring one.

This reduces the search space dramatically without losing the optimal solution.

Decoding Output

Moses can output:

- **1-best**: highest scoring translation
- **N-best list**: top-N translations with scores (useful for reranking)
- **Word lattice**: compact representation of all translations

Reordering Constraints in Decoding

Problem: allowing arbitrary reordering is computationally expensive.

Distortion Limit (Moses)

Moses limits reordering to within d positions (default: $d = 6$):

$$|e_{\text{start}} - f_{\text{prev\_end}}| \leq d \tag{24}$$

- $d = 0$: fully monotonic (fastest, worst quality)
- $d = 6$: standard default (good balance)
- $d = \infty$: unlimited reordering (slowest)

For Myanmar–English

Myanmar (SOV) to English (SVO) requires significant reordering. Use larger distortion limit ($d \geq 10$) or switch to Hiero for long-distance reordering.

Evaluation: Beyond BLEU

TER (Snover et al., 2006)

Translation Edit Rate: number of edits (insertions, deletions, substitutions, shifts) needed to convert MT output to reference, divided by reference length.

Lower TER = better.

METEOR (Banerjee & Lavie, 2005)

Metric for Evaluation of Translation with Explicit Ordering. Uses unigram F-score + word alignment + penalty for reordering.

Better correlation with human judgments than BLEU.

chrF (Popovic, 2015)

Character n-gram F-score: better for morphologically rich languages (Myanmar, Turkish).

Human Evaluation

- **Adequacy**: is the meaning preserved?
- **Fluency**: is the output grammatical?
- **Direct Assessment (DA)**: rate from 0–100
- **MQM**: error-type annotation (WMT standard)

Moses: Open-Source SMT

Moses (Koehn et al., 2007)

Moses is the most widely used open-source Statistical Machine Translation system. Named after the biblical figure who "led people across boundaries."

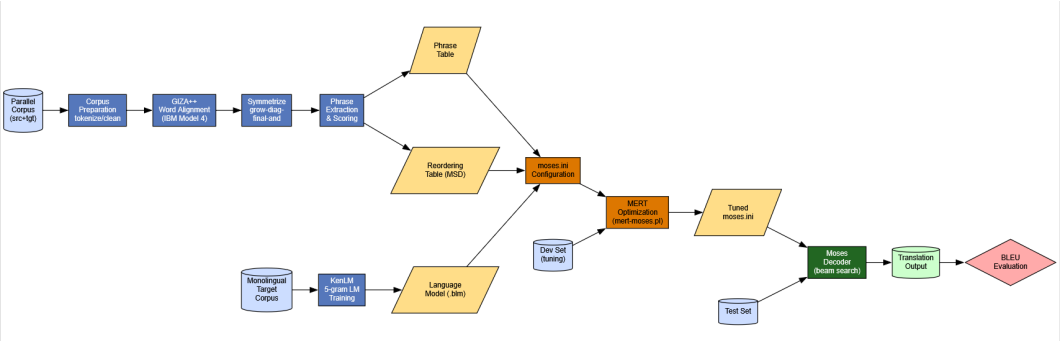
History:

- **2002**: Pharaoh decoder (phrase-based precursor)
- **2005**: Moses started by Hieu Hoang & Philipp Koehn
- **2006**: JHU summer workshop (major extension)
- **2009**: Tree-based models added
- **2012**: MosesCore EU project
- Still actively maintained on GitHub

Key Features

- Phrase-based, Hierarchical (Hiero), and Syntax-based decoding
- MERT, PRO, MIRA optimization
- Factored translation models
- On-disk phrase tables (memory-efficient)
- Multiple language model support (KenLM, SRILM)
- Neural network integration (via nplm)

Moses: External Components



Moses: External Components

Word Alignment:

- GIZA++ (IBM Models)
- fast_align (faster, similar quality)
- Berkeley Aligner (discriminative)

Language Model:

- KenLM (default, fast)
- SRILM (more options)
- RandLM (randomized)

Evaluation:

- mteval-v13a.pl (BLEU)
- TER-plus
- Multi-BLEU

Other Tools:

- Tokenizers (many languages)
- sentence aligner (Hunalign)
- Corpus cleaning scripts

Moses: Complete Training Example

Step 1: Prepare data

```
# Tokenize
tokenizer.perl -l en < train.en \
  > train.tok.en
# Lowercase
lowercase.perl < train.tok.en \
  > train.lc.en
# Clean (max 80 words)
clean-corpus-n.perl train.lc my en \
  train.clean 1 80
```

Step 2: Train LM

```
lmplz -o 5 < train.lc.en \
  > lm.arpa
build_binary lm.arpa lm.blm
```

Step 3: Train model

```
perl train-model.perl \
  -root-dir model \
  -corpus train.clean \
  -f my -e en \
  -alignment \
    grow-diag-final-and \
  -reordering \
    msd-bidirectional-fe \
  -lm 0:5:lm.blm:8 \
  -mgiza \
  -external-bin-dir tools
```

Moses: Tuning and Decoding

Step 4: Tune weights (MERT)

```
perl mert-moses.pl \
  dev.my dev.en \
  /path/to/moses \
  model/model/moses.ini \
  --mertdir /tools \
  --decoder-flags "-threads 4"
```

Step 5: Decode test set

```
moses -f tuned.moses.ini \
  < test.tok.my \
  > test.out.en
```

Step 6: Evaluate

```
perl multi-bleu.perl \
  test.ref.en \
  < test.out.en
# Output:
# BLEU = 27.34, ...
```

Tip

Always **detokenize** output before showing to users. Use `detokenizer.perl` from Moses scripts.

Other Open-Source SMT Systems

Joshua (JHU)

- Hierarchical & syntax-based MT in **Java**
- `http://joshua.sourceforge.net`
- Good for Hiero and tree-based models

cdec (UMD)

- C++ decoder for many translation formalisms
- Supports CRF training (MIRA)
- `http://cdec-decoder.org`

Jane (RWTH Aachen)

- Phrase-based and hierarchical MT
- Good for discriminative training
- Used in shared tasks

Today's Reality

All major SMT toolkits have been superseded by Neural MT. Moses and Joshua are still used for **low-resource** and **baseline comparison** purposes.

SMT Summary

What We Learned about SMT

1. **Noisy channel model**: decompose into Translation Model + Language Model
2. **IBM Models 1–4**: word alignment via EM algorithm
3. **Phrase-based SMT**: translate phrases, not words; use log-linear features
4. **Hierarchical SMT**: hierarchical rules for long-range reordering
5. **OSM**: joint model of translation + reordering operations
6. **Language Model**: n-gram with Kneser-Ney smoothing
7. **MERT**: optimize feature weights for BLEU
8. **Moses**: the standard open-source SMT toolkit

Now we shift to Neural Machine Translation...

Why Neural Machine Translation?

Limitations of SMT:

- Separate components (TM, LM, reordering) not jointly optimized
- Phrase table grows with data (memory)
- Limited morphology handling
- No deep semantic understanding
- Feature engineering required

NMT Promise

- **End-to-end** training
- **Dense** word representations
- **Joint** modeling of all aspects
- Automatic feature learning
- Better fluency

NMT Timeline

2014	Cho et al. (encoder-decoder RNN); Sutskever et al. (seq2seq)
2015	Bahdanau et al. (attention mechanism) — breakthrough!
2016	NMT wins WMT shared tasks (most language pairs)
2017	Vaswani et al. (Transformer) — dominates NMT
2018+	BERT, mBART, T5 — pretrained language models
2022+	LLMs (ChatGPT, Gemini) achieve near-human MT

Neural Networks: Quick Review

A neural network:

- Takes **numerical inputs** (e.g., word vectors)
- Applies **linear transformations** (weights W, bias b)
- Applies **non-linear activations** (ReLU, sigmoid, tanh)
- Produces **outputs** (e.g., translation probabilities)

Layer computation:

$$\mathbf{h} = f(W\mathbf{x} + \mathbf{b}) \tag{25}$$

where f is an activation function.

Training: **backpropagation** + gradient descent minimizes cross-entropy loss.

Word Embeddings

Words are mapped to **dense vectors** (embeddings) in a continuous space:

- "king" $\approx$ [0.2, 0.8, -0.3, ...]
- Similar words are close in embedding space
- Typical dimension: 256–1024

Famous: **word2vec** (Mikolov et al., 2013)
king $-$ man $+$ woman $\approx$ queen

From SMT to NMT: Key Differences

Aspect	**SMT**	**NMT**
Architecture	Pipeline (TM + LM + Decoder)	Single end-to-end network
Word representation	Discrete (one-hot)	Dense vectors (embeddings)
Training objective	BLEU (via MERT)	Cross-entropy (likelihood)
Memory	Large phrase table	Fixed-size model parameters
Long-range	Limited	Better (via attention)
Low-resource	Competitive	Struggles (needs more data)
Morphology	Poor	Better (subword units)
Speed	Fast decoding	Slower (GPU needed)
Interpretability	High (phrase table)	Low (black box)

NMT History: Early Attempts

- **1990s**: Early neural MT attempts
  - Waibel et al. (1991): connectionist models for speech translation
  - Forcada and Ñeco (1997): RNN-based MT (strikingly similar to modern approaches!)
  - But: insufficient data, insufficient compute $\Rightarrow$ abandoned
- **2007**: Schwenk proposes **neural language models** for SMT
  - Feed-forward NN-LM improves SMT quality significantly
  - Large improvement in evaluation campaigns
- **2012–2013**: Neural features integrated into SMT components (Devlin et al., Schwenk, others)
- **2013**: Kalchbrenner & Blunsom — CNN-based encoder-decoder
- **2014**: **Cho et al. and Sutskever et al.** — full neural MT breakthrough
- **2015**: **Bahdanau et al.** — attention mechanism resolves long-sentence problem
- **2016**: NMT wins WMT in nearly all language pairs
- **2017**: **Transformer** makes NMT much faster and better

NMT Toolkits Overview

Toolkit	Framework	Language	Notes
OpenNMT-py	PyTorch	Python	Very popular, flexible
OpenNMT-tf	TensorFlow	Python	Scalable
Marian NMT	Custom	C++	Very fast, production ready
Fairseq	PyTorch	Python	Facebook/Meta, supports all
Nematus	Theano/MXNet	Python	Original attention NMT
JoeyNMT	PyTorch	Python	Educational, clean code
Sockeye	MXNet/PyTorch	Python	Amazon
HuggingFace	PyTorch/TF	Python	Transformer models

Recommendation for Beginners

Start with **OpenNMT-py** or **JoeyNMT** for learning. Use **Marian NMT** for production/speed. Use **Fairseq** for large-scale research.

Cho et al. (2014) — The Breakthrough Paper

Paper

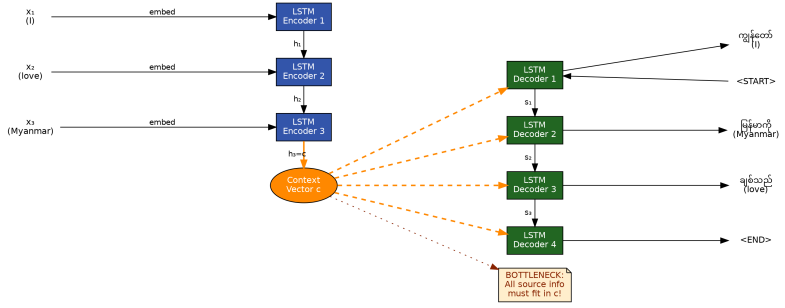
Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014).
"Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation."
EMNLP 2014.

- Proposed the **RNN Encoder-Decoder** (Seq2Seq) architecture
- Variable-length input → **fixed-length context vector** → variable-length output
- Also introduced the **GRU (Gated Recurrent Unit)**
- Initially used as an **additional feature** for SMT; later used standalone

Simultaneously

Sutskever et al. (2014) at Google proposed a similar "Sequence to Sequence" model using **LSTM** cells — another landmark paper published at NIPS 2014.

RNN Encoder-Decoder Architecture



Encoder RNN:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t) \tag{26}$$

Reads source sentence $\mathbf{x}$ step by step, outputs final hidden state $\mathbf{c} = \mathbf{h}_T$.

Decoder RNN:

$$\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{y}_{t-1}, \mathbf{c}) \tag{27}$$

$$P(y_t) = \text{softmax}(W\mathbf{s}_t + b) \tag{28}$$

Generates target words conditioned on $\mathbf{c}$ and previous output.

Gated Recurrent Units (GRU)

GRU (Cho et al., 2014)

A simplified alternative to LSTM with **two gates**:

$$\mathbf{r}_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1}) \quad \text{(Reset gate)} \tag{29}$$

$$\mathbf{z}_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1}) \quad \text{(Update gate)} \tag{30}$$

$$\tilde{\mathbf{h}}_t = \tanh(W \mathbf{x}_t + U(\mathbf{r}_t \odot \mathbf{h}_{t-1})) \quad \text{(Candidate)} \tag{31}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad \text{(Output)} \tag{32}$$

Symbols: σ = sigmoid, $\odot$ = element-wise product (Hadamard)
Reset gate r_t: how much of previous state to forget
Update gate z_t: how much of candidate to mix in

GRU vs. LSTM

GRU has **fewer parameters** than LSTM (no separate cell state), trains faster, and performs similarly. LSTM is slightly better for very long sequences.

The Bottleneck Problem

Critical Weakness

The entire source sentence meaning must be compressed into a **single fixed-size context vector** $\mathbf{c}$.

For long sentences, this **bottleneck** causes information loss and translation quality degrades significantly.

Evidence (Cho et al., 2014):

- Translation quality is good for short sentences (up to 20 words)
- Quality drops sharply for sentences > 30 words
- The fixed-size vector simply cannot hold all information

Solution

Bahdanau et al. (2015) introduced the **attention mechanism** to solve this bottleneck: instead of one vector, use **all encoder hidden states** and **selectively focus** on relevant parts.

Training NMT: Maximum Likelihood

Training Objective

Minimize **cross-entropy** (equivalent to maximizing log-likelihood):

$$\mathcal{L} = -\sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \sum_{t=1}^{T} \log P(y_t \mid y_1, \ldots, y_{t-1}, \mathbf{x}) \tag{33}$$

Train using **teacher forcing**: feed the correct previous target word at each decoder step (not the model's own prediction) during training.

- Optimizer: **Adam** (adaptive learning rate, very common in NMT)
- Learning rate: 0.0001 – 0.001 typical
- Batch size: 32–256 sentence pairs
- Epochs: 10–50 depending on dataset size
- Early stopping based on validation BLEU or loss

Key: Parallel Data Required

NMT needs aligned sentence pairs. Quality and quantity both matter!

LSTM: Long Short-Term Memory

LSTM (Hochreiter & Schmidhuber, 1997)

LSTMs solve the **vanishing gradient problem** of simple RNNs through a **cell state** $\mathbf{c}_t$ and three gates:

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + b_f) \quad \text{(Forget gate: what to discard)} \tag{34}$$

$$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + b_i) \quad \text{(Input gate: what to remember)} \tag{35}$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + b_c) \quad \text{(Candidate cell state)} \tag{36}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{(Cell state update)} \tag{37}$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + b_o) \quad \text{(Output gate)} \tag{38}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{(Hidden state output)} \tag{39}$$

Key: cell state $\mathbf{c}_t$ flows through time with only **element-wise operations** — gradient flows easily (highway).

Bidirectional LSTM (BiLSTM)

Schuster & Paliwal (1997) — Bidirectional RNNs

A **BiLSTM** processes the input sequence in **both directions**:

$$\overrightarrow{\mathbf{h}}_t = \text{LSTM}(\overrightarrow{\mathbf{h}}_{t-1}, \mathbf{x}_t) \quad \text{(Forward)} \tag{40}$$

$$\overleftarrow{\mathbf{h}}_t = \text{LSTM}(\overleftarrow{\mathbf{h}}_{t+1}, \mathbf{x}_t) \quad \text{(Backward)} \tag{41}$$

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \quad \text{(Concatenate)} \tag{42}$$

Advantage: Each position sees **context from both left and right**. This is crucial for understanding ambiguous source words.

Myanmar Example (word disambiguation)

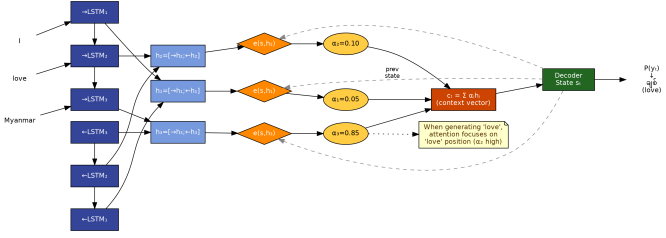
ကျွန်တော် ဘဏ်သွားမည် (I will go to the bank)
"bank" (river/financial) — BiLSTM sees full sentence context for disambiguation.

The Attention Mechanism (Bahdanau et al., 2015)

Paper

Bahdanau, D., Cho, K., & Bengio, Y. (2015).
"Neural Machine Translation by Jointly Learning to Align and Translate."
ICLR 2015. — One of the most cited NLP papers ever!

Key idea: Instead of using a single context vector, let the decoder **"attend"** to different encoder positions at each decoding step.



Attention Mechanism: The Math

Computing Attention

At each decoder step t:

$$e_{tj} = a(\mathbf{s}_{t-1}, \mathbf{h}_j) \quad \text{(alignment score: how much does } \mathbf{h}_j \text{ match } \mathbf{s}_{t-1}\text{?)} \tag{43}$$

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{T_x} \exp(e_{tk})} \quad \text{(softmax normalization: attention weights)} \tag{44}$$

$$\mathbf{c}_t = \sum_{j=1}^{T_x} \alpha_{tj} \mathbf{h}_j \quad \text{(context vector: weighted sum of encoder states)} \tag{45}$$

Alignment function $a(\cdot, \cdot)$ (Bahdanau style):

$$a(\mathbf{s}_{t-1}, \mathbf{h}_j) = \mathbf{v}_a^\top \tanh(W_a \mathbf{s}_{t-1} + U_a \mathbf{h}_j) \tag{46}$$

$\mathbf{v}_a$, W_a, U_a are **learned parameters** (the attention model).

Attention Weights —Intuition

What attention weights α_{tj} mean:

- $\alpha_{tj} \approx 1$: decoder position t strongly attends to encoder position j
- $\alpha_{tj} \approx 0$: position j is ignored at step t
- The attention matrix can be **visualized** as a heat map
- Provides **interpretability**: we can see which source words influenced each target word

Key Benefit

Attention effectively learns **soft word alignment** simultaneously with translation — no separate GIZA++!

Myanmar-English Attention

Translating "ကျွန်တော် ကွန်ပျူတာ သုံးသည်" → "I handle the computer"

When generating "handle", attention focuses on သုံး

When generating "computer", attention focuses on ကွန်ပျူတာ

Full BiLSTM NMT Architecture

Complete model (Bahdanau et al., 2015):

1. **Embedding**: map source words to dense vectors $\mathbf{x}_t$
2. **BiLSTM Encoder**: compute $\mathbf{h}_j = [\overrightarrow{\mathbf{h}}_j; \overleftarrow{\mathbf{h}}_j]$ for all positions j
3. **Attention**: compute context vector $\mathbf{c}_t$ for each decoder step t
4. **LSTM Decoder**: compute state $\mathbf{s}_t = f(\mathbf{s}_{t-1}, \mathbf{y}_{t-1}, \mathbf{c}_t)$
5. **Output projection**: $P(y_t) = \text{softmax}(W_o \mathbf{s}_t)$

Key Results (WMT English-French)

BiLSTM with attention achieved BLEU of **28.45**, outperforming all SMT baselines at the time (2015). For the first time, a pure neural model was competitive!

Remaining Issue

Training is slow; long sequences still challenging. Solved by **Transformer (2017)**.

Beam Search in NMT Decoding

NMT Decoding

Find the most likely target sequence:

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \sum_{t=1}^{T} \log P(y_t \mid y_1, \ldots, y_{t-1}, \mathbf{x}) \tag{47}$$

Greedy decoding: always pick $\arg\max P(y_t| \ldots)$ at each step. Beam search: maintain top-b partial hypotheses.

NMT Beam Search:

- Beam size $b = 5$ typical (much smaller than SMT)
- Hypotheses terminated by </s> token
- **Length normalization**: divide score by output length $^{\alpha}$ (prevents short outputs)
- **Coverage penalty**: ensure all source words are attended to

```
# Beam search (conceptual)
beam = [("", 0.0)]
for step in range(max_len):
  candidates = []
  for hyp, score in beam:
    next_probs = model.decode(hyp)
    for w, p in top_k(next_probs):
      candidates.append(
        (hyp + w, score + log(p)))
  beam = sorted(candidates)[:b]
```

"Attention is All You Need" (2017)

Vaswani et al. (2017) — NIPS 2017

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., & Polosukhin, I.
"Attention is All You Need."
Advances in Neural Information Processing Systems (NIPS) 2017.

Key innovation: Replace all RNNs with **self-attention**!

Why discard RNNs?

- RNNs must process **sequentially** (slow training)
- Hard to parallelize on GPUs
- Long-range dependencies are difficult
- Gradient flow issues even with LSTM

Transformer advantages:

- **Fully parallelizable**: all positions processed simultaneously
- Direct connections between all positions
- Captures long-range dependencies easily
- Much faster training
- Scales to larger models

Self-Attention: The Core Idea

Self-Attention (Scaled Dot-Product Attention)

Each word in the sequence **attends to all other words** to compute its representation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{48}$$

- Q (Query): what this position is looking for
- K (Key): what each position offers
- V (Value): the actual content at each position
- d_k: dimension of key vectors (scaling prevents vanishing gradients)

Intuition: Dot product $QK^\top$ measures **similarity** (how relevant each key is to the query). Softmax converts to weights. V is the weighted sum = the output.

Example

For the word "bank" in "I went to the bank to deposit money", self-attention helps it look at "deposit" and "money" to resolve ambiguity.

Multi-Head Attention

Multi-Head Attention

Run h **attention heads** in parallel, each with different learned projections:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{49}$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{50}$$

$W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{model} \times d_k}$: learned projection matrices for head i

Why multiple heads?

- Different heads can attend to different **types of relationships**:
  - Head 1: syntactic dependencies (subject-verb)
  - Head 2: semantic relationships (antecedents of pronouns)
  - Head 3: positional relationships
- $h = 8$ **heads** in the original paper, $d_k = d_{model}/h = 64$

Positional Encoding

Problem: Self-attention is **permutation-invariant** — it does not know the position of each word!

Solution: Positional Encoding

Add **positional signals** to word embeddings before feeding to the Transformer:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{51}$$

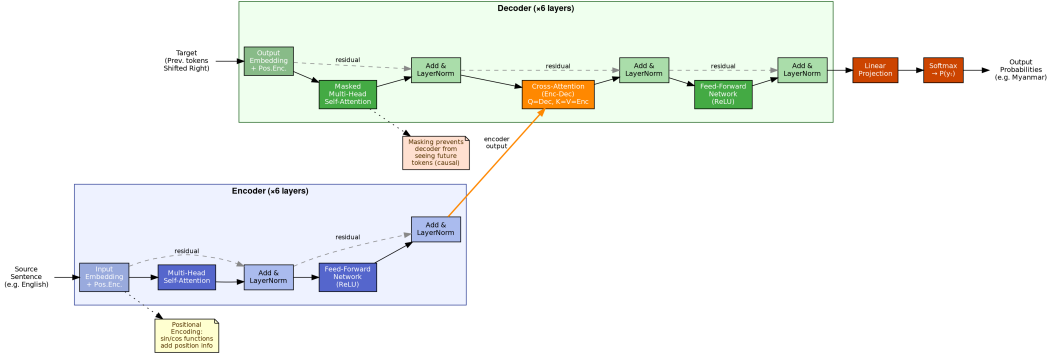
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{52}$$

where pos is the position index and i is the dimension index.

Why sinusoidal?

- Different frequencies encode different positional information
- Model can generalize to longer sequences than seen in training
- Relative positions can be represented as linear combinations
- **Alternative**: learned positional embeddings (also common in practice)

Transformer Architecture



Transformer: Encoder Sublayers

The Transformer **encoder** consists of $N = 6$ identical layers, each with:

Encoder Layer

1. **Multi-Head Self-Attention sublayer**:
   $\mathbf{h} = \text{LayerNorm}(\mathbf{x} + \text{MultiHead}(\mathbf{x}, \mathbf{x}, \mathbf{x}))$
2. **Feed-Forward sublayer**:
   $\mathbf{h}' = \text{LayerNorm}(\mathbf{h} + \text{FFN}(\mathbf{h}))$
   $\text{FFN}(\mathbf{h}) = \max(0, \mathbf{h}W_1 + b_1)W_2 + b_2$ (ReLU, inner dim=2048)

Residual connections + **Layer Normalization** at each sublayer:

- Residual: output $= \mathbf{x} + \text{sublayer}(\mathbf{x})$ (prevents vanishing gradients in deep networks)
- LayerNorm: normalizes each sample's activations (stable training)

Transformer: Decoder

The Transformer **decoder** also has $N = 6$ layers, each with **three** sublayers:

Decoder Layer

1. **Masked Multi-Head Self-Attention**:
   Attends to previous target positions only (causal masking prevents "peeking at the future")
2. **Cross-Attention (Encoder-Decoder Attention)**:
   Q from decoder, K and V from encoder output.
   This is how the decoder "reads" the source sentence.
3. **Feed-Forward sublayer** (same as encoder)

Final output: linear layer + softmax over vocabulary $\Rightarrow P(y_t)$

Key Hyperparameters

Base model: $d_{model} = 512$, $h = 8$, $d_{ff} = 2048$, $N = 6$, 65M params
Big model: $d_{model} = 1024$, $h = 16$, $d_{ff} = 4096$, $N = 6$, 213M params

Transformer Training Details

Optimizer: Adam with custom learning rate schedule:

$$lr = d_{model}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5}) \quad (53)$$

- Warmup steps: 4000
- Learning rate **increases** linearly during warmup
- Then **decreases** proportionally to $\text{step}^{-0.5}$

Regularization:

- **Dropout**: $p = 0.1$ on all sublayers
- **Label smoothing**: $\epsilon = 0.1$ (soft targets)

Results (WMT 2014)

Model	BLEU (En-De)
Best SMT	20.9
GNMT (Google)	26.3
Transformer (base)	27.3
Transformer (big)	**28.4**

Trained in **12 hours** on 8 P100 GPUs! (GNMT took weeks)

Byte Pair Encoding (BPE) for Subwords

BPE (Sennrich, Haddow, Birch, 2016)

Problem: Out-of-vocabulary (OOV) words; large vocabularies.
Solution: Represent words as sequences of **subword units**.

BPE Algorithm:

1. Start with character-level vocabulary
2. Repeatedly merge the most frequent **adjacent pair** of symbols
3. Stop after N merges (e.g., $N = 32{,}000$)

BPE Example

"*Myanmar*" $\rightarrow$ "`Myan@@ mar`" (2 subwords)
"*untranslatable*" $\rightarrow$ "`un@@ trans@@ lat@@ able`"

Key: Even unseen words can be represented! Character "`@@`" indicates continuation (not end of word).

SentencePiece (Kudo & Richardson, 2018): language-independent alternative to BPE (no tokenization needed); widely used for multilingual models.

Transformer for Myanmar Languages

Why Transformer works well for Myanmar:

- Myanmar has complex morphology; BPE handles this well
- Self-attention captures long-range dependencies
- Myanmar SOV $\leftrightarrow$ English SVO: cross-attention handles reordering
- Syllable-level tokenization + BPE is effective

Related work (Ye Kyaw Thu et al.):

- NMT for Myanmar–Rakhine (NAACL 2019)
- NMT for Myanmar dialects (2020)
- Thai–Myanmar–English NMT (TALLIP 2024)
- Improving NMT with POS-tag features (Heliyon 2022)

```
# OpenNMT-py training
python train.py \
  -data data/my-en \
  -save_model models/my-en \
  -world_size 1 \
  -gpu_ranks 0 \
  -train_steps 100000 \
  -valid_steps 5000 \
  -encoder_type transformer \
  -decoder_type transformer \
  -enc_layers 6 \
  -dec_layers 6 \
  -heads 8 \
  -transformer_ff 2048 \
  -dropout 0.1
```

Back-Translation: Using Monolingual Data

Sennrich, Haddow, & Birch (2016) — ACL 2016

Back-translation (BT): use a target→source system to translate monolingual target data, then use it as additional parallel training data.

$$\textit{Monolingual English} \xrightarrow{\text{reverse MT (en}\rightarrow\text{my)}} \textit{Synthetic Myanmar} \Rightarrow \text{Add as training pair (syn\_my, real\_en)}$$

Why BT is powerful:

- Billions of monolingual sentences available
- Dramatically improves NMT, especially for low-resource
- Helps the model learn better representations of target language
- Easy to implement (just run inference backward)

Result

BT can improve BLEU by **3–10 points** for low-resource language pairs. Essential for Myanmar ethnic language NMT where parallel data is scarce.

Marian NMT: Fast Neural MT

Marian NMT (Junczys-Dowmunt et al., 2018)

Marian is a C++ framework for neural MT, initially a reimplementation of Nematus, now supporting the full Transformer architecture. Named after **Marian Rejewski**, Polish mathematician who helped break Enigma.

Key features:

- Very **fast training and inference** (C++ backend)
- **GPU and CPU** support
- Self-contained: minimal dependencies
- Supports **Transformer, RNN, CNN** architectures
- **Multi-GPU** and multi-node training
- **Quantization** for fast CPU deployment
- Used in production at Microsoft Translator

Official Resources

- Website: `https://marian-nmt.github.io`
- GitHub: `https://github.com/marian-nmt/marian`
- Paper: Junczys-Dowmunt et al., ACL 2018

Marian NMT: Training

Preprocess with SentencePiece:

```
spm_train --input=train.en,train.my \
  --model_prefix=sp \
  --vocab_size=32000 \
  --character_coverage=0.9995 \
  --model_type=bpe

spm_encode --model=sp.model \
  --output_format=piece \
  < train.en > train.sp.en
```

Train Transformer:

```
marian --type transformer \
  --train-sets train.sp.my \
               train.sp.en \
  --valid-sets dev.sp.my dev.sp.en \
  --vocabs sp.vocab sp.vocab \
  --mini-batch-fit \
  --workspace 7000 \
  --model model.npz \
  --early-stopping 10
```

Decode/Translate:

```
cat test.sp.my | \
marian-decoder \
  --model model.npz \
  --vocabs sp.vocab sp.vocab \
  --beam-size 5 \
  --normalize 1 \
  > test.out.sp.en

# Post-process
spm_decode --model=sp.model \
  < test.out.sp.en > test.out.en
```

Tip

Use `--fp16` for half-precision training (2x faster, same quality).

Marian vs. Moses: A Comparison

Aspect	Moses (SMT)	Marian (NMT)
Approach	Phrase-based statistical	Transformer neural
Language	C++, Perl scripts	C++
Training data	Small (good for low-resource)	Needs more data
Training time	Hours on CPU	Hours on GPU
Inference speed	Very fast (CPU)	Fast (GPU), moderate (CPU)
Translation quality	Good for structured text	Better fluency overall
Interpretability	High (phrase table)	Low (black box)
Memory	Large phrase table	Fixed model file
Morphology	Poor	Better (BPE/SP)
Low-resource	Competitive	Struggles

Recommendation

Use **Moses** for very low-resource or when interpretability matters. Use **Marian** for medium-to-large data and best quality.

From NMT to Pretrained Language Models

The New Paradigm: Pretrain then Fine-tune

Rather than training from scratch for each language pair, modern MT uses:

1. **Pretraining**: train a large model on massive multilingual data to learn general language representations
2. **Fine-tuning**: adapt the pretrained model to a specific translation direction using parallel data

Key advantages:

- **Transfer learning**: knowledge from high-resource languages transfers to low-resource languages
- **Less parallel data** needed for fine-tuning
- **Better representations**: pretrained on orders of magnitude more text than any MT training set
- **Zero-shot** and **few-shot** translation possible

Word Embeddings: Foundation of Neural NLP

Word2Vec (Mikolov et al., 2013)

Train shallow neural network to predict **context words** from a center word (Skip-gram) or vice versa (CBOW). The hidden layer weights become the embeddings.

- Famous analogy: "king" − "man" + "woman" ≈ "queen"
- Dense 300-dimensional vectors for words
- Captures semantic and syntactic similarity

GloVe (Pennington et al., 2014)

- Global vectors: factorize word co-occurrence matrix
- Fast training, excellent embeddings

fastText (Bojanowski et al., 2017)

- Subword-level (character n-grams)
- Works for OOV words
- Good for **Myanmar** (morphologically rich)

Key Limitation

Word2Vec/GloVe/fastText give **one static embedding per word**. "bank" (river/financial) has the same embedding regardless of context.
⇒ Solved by **contextualized embeddings** (BERT, ELMo).

BERT and Multilingual BERT (mBERT)

BERT (Devlin et al., 2019)

Bidirectional Encoder Representations from Transformers:

- Transformer encoder pretrained on English Wikipedia + Books
- Two pretraining tasks: **Masked LM** (MLM) and **Next Sentence Prediction** (NSP)
- MLM: randomly mask 15% of tokens; predict masked tokens from context
- Creates **contextualized** word representations

mBERT (Devlin et al., 2019)

Multilingual BERT: trained on **104 languages** (Wikipedia) with a shared vocabulary of 110k subword tokens.

- Supports **Myanmar language**!
- Surprisingly good zero-shot cross-lingual transfer
- 110k vocabulary; 12 Transformer layers; 179M parameters
- Fine-tune for MT: add decoder, train on parallel data

XLM-R: Cross-Lingual Language Model

XLM-R (Conneau et al., 2020)

XLM-RoBERTa: trained on 100 languages using **Masked Language Modeling only** (better than NSP), on **2.5 TB of CommonCrawl data**.

- Significantly outperforms mBERT on all cross-lingual benchmarks
- Available in Base (270M) and Large (560M) sizes
- Key improvement: much more data + better multilingual balance
- Good for fine-tuning on low-resource language pairs
- Supports **Myanmar** (included in 100 languages)

Fine-tuning XLM-R for MT (Conceptual)

```
from transformers import XLMRobertaModel, XLMRobertaTokenizer
tokenizer = XLMRobertaTokenizer.from_pretrained("xlm-roberta-base")
model = XLMRobertaModel.from_pretrained("xlm-roberta-base")
# Add a cross-attention decoder on top
# Fine-tune on parallel corpus
```

mBART: Multilingual Denoising Pretraining

mBART (Liu et al., 2020)

Multilingual BART: sequence-to-sequence model pretrained with **denoising autoencoding** on **25 languages**.

- Based on BART architecture: Transformer encoder + decoder
- Pretraining: corrupt input (masking, deletion, permutation) $\rightarrow$ reconstruct original
- Supports both **encoder and decoder** $\Rightarrow$ ideal for MT
- mBART-50: extended to 50 languages (including Myanmar)
- Fine-tune for any language pair (even zero-shot between seen languages)

Why mBART is good for MT

Unlike BERT (encoder only), mBART has a full encoder-decoder, so fine-tuning for MT is natural. Just add source and target language tokens: `[MY_LANG] source sentence [EN_LANG]` $\rightarrow$ `target sentence`

T5: Text-to-Text Transfer Transformer

T5 (Raffel et al., 2020)

T5 (Text-to-Text Transfer Transformer, Google): frames **all NLP tasks** as text-to-text problems.

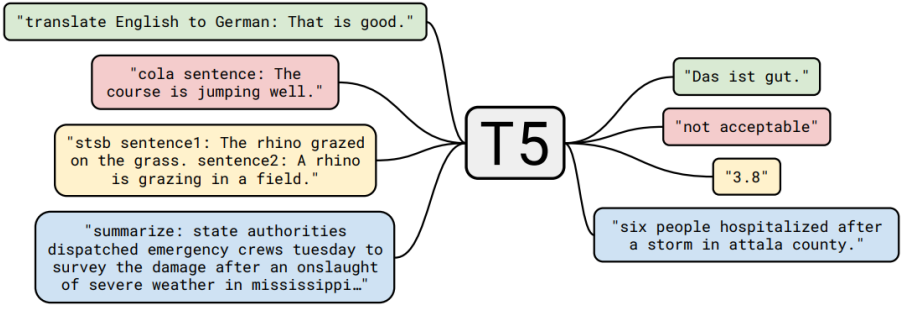
- Trained on **750 GB of clean web text** (C4 dataset)
- For translation: input = "translate English to Myanmar: {sentence}"
- Output = Myanmar translation
- Available in Small (60M) to 11B parameter versions

mT5 (Xue et al., 2021)

Multilingual T5: T5 trained on **101 languages** (mC4 dataset).

- 300M – 13B parameters
- Includes Myanmar language
- Fine-tune with: "`translate my to en:` မြန်မာ → Myanmar"

T5: Text-to-Text Transfer Transformer



Text-to-**T**ext **T**ransfer **T**ransformer

Overview of the text-to-text paradigm. By framing different NLP tasks as text-to-text problems, the model simplifies the development pipeline. The same architecture, loss function, and hyperparameters are applied across all tasks. (Source: Raffel et al., 2019)

T5 Unified Framework

MT, summarization, QA, sentiment all use the same model format. Great for **multi-task learning** (one model for many tasks).

M2M-100: Many-to-Many MT

M2M-100 (Fan et al., 2020, Meta AI)

M2M-100: first MT model trained for **any-to-any** translation across **100 languages** (10,000 language pairs).

- 1.2B parameters (large model)
- Does not require English as a pivot language
- Training data: 7.5B sentence pairs (web-mined + curated)
- Uses language tokens: `__my__` for Myanmar

Why non-English pivoting matters:

- Many languages are related but one or both are not English (e.g., Myanmar↔Thai)
- Direct translation is more accurate than pivot through English
- Fewer error propagations

Usage

```
from transformers import
     M2M100ForConditionalGeneration, \
    M2M100Tokenizer
model = M2M100ForConditionalGeneration\
    .from_pretrained("facebook/m2m100_418M")
tokenizer = M2M100Tokenizer\
    .from_pretrained("facebook/m2m100_418M")
tokenizer.src_lang = "my"
```

NLLB: No Language Left Behind —Introduction

NLLB (Costa-jussà et al., 2022, Meta AI)

NLLB-200: Meta AI's landmark multilingual MT model supporting **200 languages**, including 149 previously underserved languages.

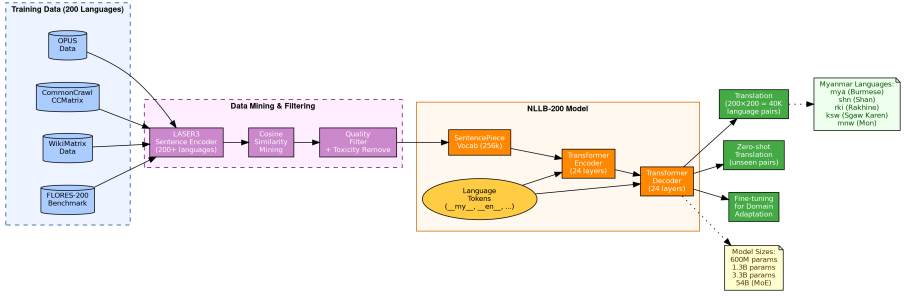
"*No Language Left Behind*" — democratizing MT for the world.

- Published: July 2022 (arXiv: 2207.04672)
- Models available: 600M, 1.3B, 3.3B parameters
- Covers **Myanmar** and several Myanmar ethnic languages
- Open-sourced weights on HuggingFace
- Part of Meta AI's effort for AI equity across languages

Impact

NLLB demonstrated that even the world's most under-resourced languages (some with $<$1M speakers) can benefit from neural MT.

NLLB: Architecture and Training



Architecture:

- Transformer encoder-decoder (Sparsely Gated MoE for 54B model)
- Shared SentencePiece vocabulary: 256k tokens
- Language-specific tokens for all 200 languages

Training data:

- OPUS, CCAligned, CCMatrix, WikiMatrix
- **Primary bitext** for 200 languages
- Data quality filtering (LASER3 filtering)
- Temperature-based sampling for balance

NLLB: Key Contributions

Technical Innovations:

1. **LASER3**: extended sentence encoder for 200+ languages (used for mining and filtering data)
2. **FLORES-200**: new benchmark for 200 languages (1012 sentence evaluation set)
3. **Sparse MoE**: Mixture of Experts routing for 54B model scales parameters without proportional compute cost
4. **Toxicity filtering**: remove harmful content from training data

NLLB Results

Language	NLLB	Prev best
Burmese	+50%	—
Shan	+70%	—
Rakhine	New!	—
Kachin	New!	—

(Relative BLEU improvements on FLORES-200)

NLLB: Myanmar and Ethnic Languages

Languages Supported (Myanmar region)

- **Burmese (mya)**: High-resource in NLLB
- **Shan (shn)**: Major ethnic language included
- **Rakhine (rki)**: Arakanese dialect included
- **Sgaw Karen (ksw)**: Karen language included
- **Mon (mnw)**: Mon-Khmer language included
- **Lahu (lhu)**, **Chin (cnh)**: also included

How NLLB helps Myanmar NLP research:

- Pre-trained weights can be **fine-tuned** on domain-specific data
- Use as strong **baseline** for comparison
- Mine parallel data with LASER3
- Transfer learning for even smaller Myanmar ethnic languages

```python
from transformers import pipeline

translator = pipeline(
  "translation",
  model="facebook/nllb-200-distilled-600M"
)
result = translator(
  "I love Myanmar", src_lang="eng_Latn",
  tgt_lang="mya_Mymr"
)
print(result[0]['translation_text'])
```

NLLB: Data Collection Pipeline

NLLB Data Collection

1. **Seed data**: start with known parallel corpora (OPUS, Flores)
2. **LASER3 encoding**: encode all sentences into multilingual embedding space
3. **Mining**: use cosine similarity to find translation pairs in web crawl
4. **Filtering**: remove low-quality pairs using a quality classifier
5. **Backtranslation**: generate synthetic parallel data for low-resource languages
6. **Training**: multi-stage training from smaller to larger model

FLORES-200 Benchmark

Evaluation set: **1012 sentences** from WikiNews, translated professionally for all 200 languages. Same evaluation set allows **fair comparison across all languages**.
Available at: `https://github.com/facebookresearch/flores`

NLLB: Fine-tuning for Domain Adaptation

```python
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
from transformers import Seq2SeqTrainer, Seq2SeqTrainingArguments

# Load pre-trained NLLB model
model_name = "facebook/nllb-200-distilled-600M"
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Fine-tune on Myanmar medical domain data
training_args = Seq2SeqTrainingArguments(
    output_dir="./nllb-my-medical",
    per_device_train_batch_size=16,
    num_train_epochs=3,
    learning_rate=5e-5,
    warmup_steps=500,
    weight_decay=0.01,
    predict_with_generate=True,
    fp16=True,  # Mixed precision for speed
)

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
)
trainer.train()
```

LLM-Based MT: ChatGPT and GPT-4

Large Language Models for MT

Modern LLMs like **ChatGPT (GPT-3.5/4)** and **Gemini** can translate without any MT-specific fine-tuning, using their general language understanding.

LLM Translation Approaches:

1. **Zero-shot**: "Translate this to Myanmar: Hello world"
2. **Few-shot**: provide 3–5 examples before the sentence
3. **Chain-of-thought**: "Think step by step and translate..."
4. **Multi-step**: first translate, then review and revise

Capabilities:

- Near-human quality for high-resource languages
- Style and tone adaptation
- Terminology consistency with prompting
- Handles ambiguous idioms better

ChatGPT MT Prompt

```
System: You are an expert Myanmar language translator.
User: Translate the following English text to Myanmar (Burmese): {text}
```

Few-shot works well for specialized domains (medical, legal).

Gemini for Machine Translation

Gemini (Google DeepMind, 2023)

Gemini is Google's multimodal LLM family:

- Ultra, Pro, Nano versions (different capabilities)
- Trained on diverse multilingual data including 74 languages
- Excellent performance on MT benchmarks
- Supports Myanmar, Thai, and several Asian languages
- Available via Google Translate API (powered partly by Gemini)

LLM Translation Quality (2023 research):

- GPT-4 achieves near-human quality for EN-DE, EN-ZH
- Gemini Ultra competitive with GPT-4
- Both excel at formal and literary translation
- Weaknesses: low-resource languages, rare idioms
- Myanmar: improving but not yet at state-of-art

Limitation

LLMs are **black boxes**: hard to control, expensive to run, no deterministic output. For production Myanmar MT, fine-tuned specialized models (NLLB, Marian) are still preferred.

Comparison: MT Systems 2024

System	Type	Languages	Quality	Cost
Moses	SMT	Any (with data)	Moderate	Low (CPU)
Marian	NMT	Any	Good	Medium (GPU)
OpenNMT	NMT	Any	Good	Medium
mBART-50	Pretrained	50	Very good	Medium
M2M-100	Pretrained	100	Very good	Medium
NLLB-200	Pretrained	200	Excellent	Medium
DeepL	Commercial	33	Excellent	API fee
Google Translate	Commercial	133	Excellent	API fee
ChatGPT/GPT-4	LLM	100+	Near-human	High
Gemini	LLM	74+	Near-human	High

For Myanmar NLP Research

NLLB-200 + fine-tuning is currently the best starting point. Use Moses as a baseline. Use ChatGPT for data augmentation (careful!).

Reinforcement Learning for MT Fine-tuning

RLHF for MT (Recent Trend)

Reinforcement Learning from Human Feedback (RLHF) can further improve MT quality by training models to produce translations preferred by humans.

- Step 1: Collect **human preferences** between MT outputs
- Step 2: Train a **reward model** to predict human preferences
- Step 3: Fine-tune the MT model with **PPO** to maximize reward

Research: RL for Myanmar Dialect MT

Ye Kyaw Thu, Thazin Myint Oo, Thepchai Supnithi (2023):
"Reinforcement Learning Fine-tuning for Improved Neural Machine Translation of Burmese Dialects" (Workshop M3Oriental, EMNLP 2023)

Applied RL fine-tuning to improve NMT between Burmese and Rakhine dialect with small parallel corpus. RL-based fine-tuning improved fluency and adequacy.

Low-Resource MT: Key Strategies

Challenge: Most Myanmar ethnic languages have very little parallel data.

Strategies for Low-Resource MT

1. **Back-translation**: generate synthetic parallel data
2. **Transfer learning**: fine-tune NLLB or mBART
3. **Data augmentation**: paraphrase, word substitution
4. **Pivot translation**: use a bridge language (e.g., Myanmar $\rightarrow$ Thai $\rightarrow$ English)
5. **Multilingual MT**: train one model for multiple related language pairs
6. **Subword/character models**: use character or syllable units for morphologically rich languages
7. **Unsupervised MT**: train with only monolingual data (Lample et al., 2018)

Myanmar Research

Ye Kyaw Thu et al. have applied most of these strategies to Myanmar ethnic language pairs including Shan, Rakhine, Kachin, Kayah, Pa'O, Chin, and Dawei.

Multilingual MT: One Model for Many Pairs

Multilingual NMT

Train a **single model** on many language pairs simultaneously:

- Add language tokens: `<2my>` (translate to Myanmar)
- Share parameters across all languages
- Enables **zero-shot transfer**: model can translate between unseen language pairs (e.g., Shan↔Rakhine if both seen with English)
- Particularly effective for **related language families**

Myanmar Research Application

Hay Man Htun et al. (2021, JIIST): Trained multilingual SMT system combining Phrase-based, Hierarchical, and OSM for Myanmar–Pa'O language pair.

Mya Ei San et al. (2024, TALLIP): Thai–Myanmar–English multilingual NMT with SwitchOut data augmentation.

MT for Special Applications

Myanmar Sign Language MT:

- **Swe Zin Moe et al. (2020)**: Unsupervised NMT between Myanmar Sign Language and Myanmar text
- **NMT for MSL**: using sequence-to-sequence models
- Challenge: sign language has its own grammar, visual modality

Myanmar Braille MT:

- **Zun Hlaing Moe et al. (2021)**: Myanmar text to Braille (Mu Thit) translation using IBM Model 1 and 2
- Braille codes mapped to IBM translation model

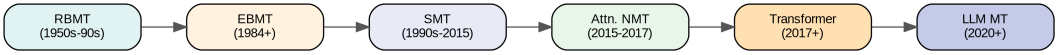
Interesting Challenges

- Sign languages are **spatial** and **visual**
- No standard written form for most sign languages
- Need specialized corpora (video, gloss)
- Myanmar Sign Language is different from ASL/BSL

Khmer Braille (2024)

Kimhuoy Yann, Ye Kyaw Thu et al. (CITA 2024): SMT vs. NMT for Khmer Braille translation.

Journey Through MT: Summary



Key lessons:

1. Rules $\rightarrow$ Statistics $\rightarrow$ Neural: data-driven beats hand-crafted
2. Better architectures: word $\rightarrow$ phrase $\rightarrow$ seq2seq $\rightarrow$ Transformer
3. Scaling works: more data and parameters consistently improve quality
4. Low-resource remains a challenge; transfer learning is the key
5. Evaluation is hard; human judgment is the gold standard

Selected Contributions to MT (Ye Kyaw Thu et al.)

SMT Research

- Phrase-based, Hierarchical, OSM for Myanmar–Pa'O (*JIIST 2021*)
- SMT for Myanmar–Rakhine, Burmese–Dawei/Tavoyan, Myanmar–Myeik (*NSURL 2019, ICCA 2020*)
- SMT for Myanmar ethnic languages: Kachin-Rawang, Kayah, Chin, Shan (*iSAI-NLP 2019, iSAI-NLP 2020, COCOSDA 2020, JIIST 2020*)
- IBM Model 1&2 for Myanmar Text (Burmese) and Braille (*JIIST 2021*)
- SMT between Myanmar Sign Language and Myanmar text (*ICCA 2018*)
- SMT between Myanmar Sign Language and Myanmar SignWriting (*ASEAN-AI 2018*)
- Large-scale SMT study for Myanmar language (*SNLP 2016*)
- A Study of SMT Methods for Under Resourced Languages (*SLTU 2016*)

Selected Contributions to MT (Ye Kyaw Thu et al.)

NMT Research

- NMT between Myanmar Sign Language and Myanmar Written Text (*ONA 2018*)
- NMT for Myanmar–Rakhine (Arakanese) (*VarDial@NAACL 2019*)
- NMT for Myanmar dialects (Dawei) (*ICCA 2020, JIIST 2020*)
- Unsupervised NMT for Myanmar Sign Language (*JIIST 2020*)
- Improving NMT with POS-tag features for low-resource pairs (*Heliyon 2022*)
- Thai–Myanmar–English multilingual NMT (*TALLIP 2024*)
- RL fine-tuning for Burmese dialect NMT (*M3Oriental@EMNLP 2023*)
- Hybrid SMT for English-Myanmar at WAT-2021 (*WAT 2021*)

Research Highlights: Prof. Ye Kyaw Thu & The LU Lab

Language Understanding Lab, Myanmar:

- Pioneered parallel corpus creation for Myanmar ethnic languages
- Developed SMT and NMT systems for previously untranslated language pairs
- Word-to-word translation between Myanmar and ethnic languages
  (May Myat Myat Khaing et al., JIIST 2022)
- English to Burmese MT with Asian pivot languages
  (Wint Theingi Zaw et al., JIIST 2022)
- Myanmar Sign Language translation
  (Hlaing Myat Nwe et al., MT Summit 2023)

Datasets Released

Most of the datasets available at:
`https://github.com/ye-kyaw-thu`

- myPar (ethnic) (ပြင်ဆင်နေဆဲ)
- Asian Language Treebank (ALT) Myanmar
- MSL4Emergency sign language corpus
- myBraille corpus (ပြင်ဆင်နေဆဲ)
- Burmese dialect corpus

Practical Session

Day 2: Hands-on SMT and NMT

We will work with parallel corpora developed by Dr. Ye Kyaw Thu.

Practical SMT (Moses):

1. Prepare Myanmar–English parallel corpus
2. Train language model (KenLM)
3. Run word alignment (GIZA++)
4. Extract phrase table
5. Tune weights (MERT)
6. Evaluate with BLEU

Practical NMT (OpenNMT-py / Marian):

1. Preprocess with SentencePiece
2. Configure Transformer model
3. Train on GPU
4. Decode test set
5. Evaluate with BLEU, chrF
6. Compare SMT vs. NMT results

Datasets to Use

Myanmar–Thai corpus from Core AI Resarch Team, NECTEC, Thailand.

Future R&D Trends in MT

Research Directions:

1. **Very low-resource MT**: 100–1000 sentence pairs
2. **Document-level MT**: context beyond sentences
3. **Multimodal MT**: text + images (visual MT)
4. **Speech-to-speech** translation (direct, no text)
5. **Real-time / simultaneous** MT (before utterance ends)
6. **Interpretable NMT**: understanding attention and decisions
7. **Sustainable MT**: reduce carbon footprint of training

Myanmar-Specific Challenges:

1. MT for endangered ethnic languages (Rawang, Palaung, Lahu)
2. Sign Language MT (Myanmar Sign Language)
3. Braille MT for accessibility
4. Dialect translation (Rakhine, Dawei, Mandalay)
5. Code-switching (Myanmar + English)
6. Myanmar→Thai for border communities
7. LLM evaluation for Myanmar language quality

Key References: SMT

- Brown, P.F., et al. (1993). *The mathematics of statistical machine translation.* Computational Linguistics, 19(2), 263–311.
- Koehn, P., Och, F.J., & Marcu, D. (2003). *Statistical phrase-based translation.* HLT-NAACL 2003.
- Och, F.J. (2003). *Minimum error rate training in statistical machine translation.* ACL 2003.
- Chiang, D. (2007). *Hierarchical phrase-based translation.* Computational Linguistics, 33(2).
- Durrani, N., et al. (2011). *A joint sequence translation model with integrated reordering.* ACL 2011.
- Koehn, P., et al. (2007). *Moses: Open source toolkit for SMT.* ACL 2007 (demo).
- Papineni, K., et al. (2002). *BLEU: a method for automatic evaluation of MT.* ACL 2002.
- Heafield, K. (2011). *KenLM: Faster and smaller language model queries.* WMT 2011.
- Koehn, P. (2009). *Statistical Machine Translation.* Cambridge University Press. (Textbook)

Key References: NMT and Pretrained Models

- Cho, K., et al. (2014). *Learning phrase representations using RNN encoder-decoder for SMT.* EMNLP 2014.
- Sutskever, I., Vinyals, O., & Le, Q.V. (2014). *Sequence to Sequence Learning with NNs.* NIPS 2014.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural MT by jointly learning to align and translate.* ICLR 2015.
- Vaswani, A., et al. (2017). *Attention is all you need.* NIPS 2017.
- Sennrich, R., et al. (2016). *Neural MT of rare words with subword units.* ACL 2016.
- Liu, Y., et al. (2020). *Multilingual denoising pre-training for NMT.* TACL 2020.
- Costa-jussà, M.R., et al. (2022). *No Language Left Behind.* arXiv:2207.04672.
- Fan, A., et al. (2021). *Beyond English-centric multilingual MT.* JMLR 2021.
- Junczys-Dowmunt, M., et al. (2018). *Marian: Fast NMT.* ACL 2018.
- Koehn, P. (2017). *Neural Machine Translation.* arXiv:1709.07809.

Resources and Useful Links

Software and Toolkits:

- Moses: http://statmt.org/moses
- KenLM: https://kheafield.com/code/kenlm
- fast_align: https://github.com/clab/fast_align
- OpenNMT: http://opennmt.net
- Marian: https://marian-nmt.github.io
- Fairseq: https://github.com/facebookresearch/fairseq
- HuggingFace: https://huggingface.co

Data and Benchmarks:

- OPUS: http://opus.nlpl.eu
- FLORES-200: https://github.com/facebookresearch/flores
- WMT: http://statmt.org/wmt
- MT-Bench: https://huggingface.co/spaces/lmsys/mt-bench

Myanmar NLP Resources of LU Lab., Myanmar:

- https://github.com/ye-kyaw-thu
- https://sites.google.com/site/yekyawthunlp

Thank You!

Questions? Comments?

ykt.nlp.ai@gmail.com

Appendix

Extra Notes & Additional Information

Why MT Evaluation is Hard

- There is **no single correct translation** —many valid translations exist
- Human evaluation is the gold standard but **expensive and slow**
- Automatic metrics enable fast feedback during development
- Key criteria for a good MT output:
  1. **Adequacy**: Does the output convey the same meaning as the source?
  2. **Fluency**: Is the output grammatically correct and natural?
  3. **Fidelity**: Are named entities, numbers, and facts preserved?

Important Caveat

No automatic metric perfectly correlates with human judgment. Always report multiple metrics and back them up with human evaluation for important decisions or research publication.

BLEU Score: Details and Calculation

Modified N-gram Precision

For each n-gram order n, compute the **modified precision** p_n:

$$p_n = \frac{\sum_{\bar{e} \in \text{output}} \min(\text{count}(\bar{e}, \text{output}),\ \max_{\text{ref}} \text{count}(\bar{e}, \text{ref}))}{\sum_{\bar{e} \in \text{output}} \text{count}(\bar{e}, \text{output})} \tag{54}$$

"Clipping": each n-gram counted at most as many times as it appears in any reference.

BLEU Step-by-Step

Reference: "I will go to Yangon tomorrow", **MT output**: "I will go Yangon tomorrow"

n-gram	Output matches	Clipped count	Total output n-grams	p_n
1	I, will, go, Yangon, tomorrow	5	5	1.00
2	"I will", "will go", "Yangon tomorrow"	3	4	0.75
3	"I will go"	1	3	0.33
4	none	0	2	0.00

$\text{BLEU} = \text{BP} \cdot \exp(0.25 \cdot (\log 1 + \log 0.75 + \log 0.33 + \log 0)) \approx 0$
(Zero because $p_4 = 0$ — the log penalty dominates)

TER, METEOR, chrF

TER (Snover et al., 2006)

Translation Edit Rate:

$$\text{TER} = \frac{\text{edits}}{\text{ref length}} \tag{55}$$

Edits = insertions + deletions + substitutions + **shifts**

Lower TER = better. Shifts allow moving phrases to correct position (free).

METEOR (Banerjee & Lavie, 2005)

Uses **flexible matching**: exact, stem, synonym
Combines precision, recall, and fragment penalty:
$F_{\text{mean}} = \frac{10 \cdot P \cdot R}{R + 9P}$
$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{frag. pen.})$
Better human correlation than BLEU.

chrF (Popovic, 2015)

Character n-gram F-score:

$$\text{chrF} = \frac{(1+\beta^2) \cdot P_{\text{chr}} \cdot R_{\text{chr}}}{\beta^2 P_{\text{chr}} + R_{\text{chr}}} \tag{56}$$

Character-level: more robust for morphologically rich languages.

chrF++ also includes word n-grams.

Recommended for Myanmar, Finnish, Turkish, Arabic.

Tip

Use BLEU + TER + chrF together for a comprehensive picture. WMT uses all three for official rankings.

COMET: Neural MT Evaluation

COMET (Rei et al., 2020)

Crosslingual Optimized Metric for Evaluation of Translation: uses a **neural model** fine-tuned on human judgments to score translations.

- Based on XLM-RoBERTa (multilingual pretrained encoder)
- Input: source sentence + MT output + reference translation
- Output: quality score (0–1, higher = better)
- Much higher correlation with human judgments than BLEU
- **COMET-22** and **BLEURT** are state-of-the-art automatic metrics

Metric Correlation with Humans

Metric	Pearson r (WMT21)
BLEU	0.63
TER	0.55
chrF	0.67
METEOR	0.64
COMET-22	**0.87**

Usage

```
pip install unbabel-comet
comet-score \
  -s source.txt \
  -t hypothesis.txt \
  -r reference.txt
```

Human Evaluation Methods

Traditional Methods:

- **Adequacy**: rate 1–5 how well meaning is preserved
- **Fluency**: rate 1–5 grammaticality of output
- **Pairwise ranking**: which of A or B is better?

Modern Methods:

- **Direct Assessment (DA)**: rate translation 0–100 independently
- **MQM (Multidimensional Quality Metrics)**: annotate specific error types
  - Accuracy, Fluency, Terminology, Style
- **SQM**: Scalar Quality Metric (Google WMT 2020+)

Evaluation for Myanmar

For Myanmar MT evaluation:

- Find native Myanmar speakers (bilingual evaluators)
- Use bilingual back-translation evaluation
- Post-edit effort studies (how much do translators have to fix?)
- FLORES-200 has Myanmar included for benchmark comparison

FLORES-200: The Standard Benchmark

FLORES (Few-shot Learning Across Languages)

FLORES-200: evaluation benchmark for 200 languages created by Meta AI.

- **1012 sentences** per language (from WikiNews)
- Split: `dev` (997 sentences) + `devtest` (1012 sentences)
- Professionally translated (not machine-generated)
- Covers all 200 NLLB languages
- Includes **Myanmar (mya)**, Shan (shn), Rakhine (rki), Karen (ksw), Mon (mnw)

How to use FLORES-200:

1. Download from GitHub
2. Translate `devtest.eng_Latn` with your system
3. Compare output against `devtest.mya_Mymr`
4. Report sacrebleu BLEU, chrF, COMET

```
pip install sacrebleu
# Compute BLEU
sacrebleu devtest.mya_Mymr \
  -i output.mya \
  -m bleu chrf

# chrF++ (recommended for Myanmar)
sacrebleu devtest.mya_Mymr \
  -i output.mya \
  -m chrf --chrf-word-order 2
```

Word Alignment Symmetrization

Symmetrization (Och & Ney, 2003)

GIZA++ gives **directional** alignments: $e \rightarrow f$ and $f \rightarrow e$. Combining them gives better coverage:

Common symmetrization heuristics:

Intersection Keep only alignments in **both** directions (high precision, low recall)

Union Keep all alignments from either direction (high recall, low precision)

grow-diag-final-and Start with intersection; iteratively add diagonal, then non-diagonal points from union that improve coverage. Default in Moses.

Moses Command

```
# After GIZA++ runs both directions:
atools -c grow-diag-final-and \
  -i fwd.A3.final \
  -j rev.A3.final \
  > sym.align
```

Or automatically via `train-model.perl`: `-alignment grow-diag-final-and`

Lexical Weighting in Phrase Table

Lexical Translation Probability

Beyond phrase probability, Moses also computes **lexical weighting** to better score rare phrases:

$$\text{lex}(\bar{f} \mid \bar{e}, a) = \prod_{i=1}^{n} \frac{1}{|\{j : a(j) = i\}|} \sum_{j:a(j)=i} w(f_j \mid e_i) \tag{57}$$

where $w(f \mid e)$ is the word-level translation probability from IBM Model 1, and a is the word alignment within the phrase pair.

- Lexical weighting helps when the **phrase count is low** (sparse data)
- A phrase seen once gets low phrase probability but can still be trusted if its component words translate well
- Both directions are computed: $\text{lex}(\bar{f}|\bar{e})$ and $\text{lex}(\bar{e}|\bar{f})$
- Stored as separate features in the phrase table (features 3 and 4)

Moses: Configuration File (moses.ini)

```
# moses.ini excerpt
[input-factors]
0

[mapping]
0 T 0

[ttable-file]
0 0 0 5 /path/to/phrase-table.gz

[lmodel-file]
8 0 5 /path/to/lm.blm

[distortion-limit]
6

[weight-t]      # phrase table weights
0.2 0.2 0.2 0.2 0.2
[weight-l]      # language model weight
0.5
[weight-d]      # distortion weight
0.3
[weight-w]      # word penalty
-1
[weight-lex]    # lexical reordering
0.3 0.3 0.3 0.3 0.3 0.3
```

Key Settings

- `distortion-limit`: max reordering distance (default 6)
- `weight-t`: 5 phrase table features
- `weight-l`: LM weight
- `weight-d`: distortion (reordering)
- `weight-w`: word penalty (negative = prefer longer output)

After MERT, these weights are automatically optimized on the dev set.

SMT for Myanmar: Practical Notes

Myanmar-specific preprocessing:

- Myanmar has **no spaces between words** ⇒ word segmentation needed!
- Use **myWord** or **syllable-based** segmentation
- Syllable segmentation is simpler and works well for SMT
- Myanmar script: each syllable is a clear unit
- Use **regex-based syllable breaking** for Myanmar ethnic languages (sylbreak4all: Ye Kyaw Thu et al., 2021)

Distortion limit for Myanmar–English:

- Myanmar is **SOV**; English is **SVO**
- Use `-distortion-limit 10` or more
- Or use **Hierarchical** (Hiero) for better reordering

Syllable Segmentation Example

Input: ကျွန်တော်သည်

After syllable break:
ကျွန် တော် သည်

(3 syllables, each is a processing unit for GIZA++)

Reference

sylbreak4all: `https://github.com/ye-kyaw-thu/sylbreak4all`
Supports 9 Myanmar ethnic languages.

Layer Normalization and Residual Connections

Why These Matter in Transformers

Deep networks (12–96 layers) are hard to train without special techniques:

- **Residual Connection** (He et al., 2016): $\mathbf{x}_{\text{out}} = \mathbf{x} + \text{sublayer}(\mathbf{x})$
  - Gradient can flow directly through the skip connection
  - Prevents vanishing gradients in very deep networks
- **Layer Normalization** (Ba et al., 2016): normalize each sample's activations to zero mean, unit variance, then apply learnable scale and shift:

$$\text{LN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sigma + \epsilon} + \beta \tag{58}$$

μ, σ: mean and std of activations within one sample; γ, β: learned parameters.

Pre-LN vs. Post-LN

Original Transformer (Vaswani 2017): Post-LN (LN after residual).
Newer practice: **Pre-LN** (LN before sublayer) — more stable training for very deep models.

Dropout and Label Smoothing in NMT

Dropout (Srivastava et al., 2014)

During training, randomly set fraction p of activations to zero. Forces the network to learn **redundant representations**.

In Transformer:

- Applied after each sublayer: $p = 0.1$
- Applied to attention weights
- Applied to embeddings

Turned off during inference.

Label Smoothing (Szegedy et al., 2016)

Instead of hard targets $y_i \in \{0, 1\}$, use **soft targets**:

$$y_i' = (1 - \epsilon)y_i + \frac{\epsilon}{K} \tag{59}$$

$\epsilon = 0.1$, $K =$ vocabulary size.

Benefits:

- Prevents overconfident predictions
- Improves calibration
- Acts as regularization
- Standard in Transformer NMT

Subword Tokenization: SentencePiece vs. BPE

BPE (Sennrich et al., 2016):

- Requires **pre-tokenization** (spaces as word boundaries)
- Language-specific tokenizers needed
- 32k–64k merge operations typical
- `@@` suffix marks continuation

SentencePiece (Kudo & Richardson, 2018):

- **Language-independent**: treats text as a raw byte stream
- No pre-tokenization needed
- Supports BPE and **unigram** model
- Space encoded as special symbol
- **Standard for multilingual NMT** (mBART, NLLB, T5)

SentencePiece for Myanmar

Input: "ကျွန်တော် မြန်မာ"

BPE output:
ကျွန် @@ တော် မြ @@ န် @@ မာ

SentencePiece output:
ကျွန်တော် မြန်မာ

(or character-level if vocab size small)

Recommendation

For Myanmar: use SentencePiece with vocab_size 8000–16000 and `character_coverage=0.9995` to handle Myanmar characters.

Transfer Learning for Low-Resource Myanmar MT

Transfer Learning Strategy

When parallel data is scarce (e.g., Myanmar–Shan with 10k sentences):

1. **Start with a pretrained multilingual model** (mBART-50, NLLB)
2. **Fine-tune** on the specific language pair
3. Much less data needed than training from scratch

Key Contributions from My Research Group:

- **Zar Zar Hlaing et al. (Heliyon, 2022)**: Adding POS-tag features to Transformer NMT improves low-resource Myanmar–Thai performance
- **Mya Ei San et al. (TALLIP, 2024)**: Thai–Myanmar–English multilingual NMT with SwitchOut augmentation
- **Thazin Myint Oo et al. (IEEE Access, 2023)**: Transfer and triangulation pivot approaches for Burmese dialects

Key Technique: Pivot

When direct parallel data (My–Ethnic) is very scarce:

My $\rightarrow$ En $\rightarrow$ Shan
(use English as pivot language)

Pivot NMT combines two models:
$P(\text{Shan}|\text{My}) \approx \sum_e P(\text{Shan}|e) \cdot P(e|\text{My})$

Or: "triangulation" directly.

Unsupervised NMT

Lample et al. (2018) — ICLR 2018

Unsupervised MT: translate between languages with **no parallel data**!

1. **Initialization**: use cross-lingual word embeddings (MUSE) to initialize the model so that similar-meaning words are close
2. **Denoising autoencoder**: train each language to reconstruct noisy input $\Rightarrow$ learns language model
3. **Back-translation (online)**: use current model to translate, then train on the generated pairs
4. **Iterate**: repeat steps 2–3 until convergence

Myanmar Application

Swe Zin Moe et al. (2020): Unsupervised NMT between Myanmar Sign Language and Myanmar Text using only monolingual data from each modality.

Challenge: very different representations (gloss sequence vs. Myanmar text)

mBART-50: Fine-tuning for Myanmar MT

mBART-50 (Tang et al., 2021)

Extension of mBART to 50 languages including Myanmar. Ideal for:

- Fine-tuning on Myanmar–English, Myanmar–Thai
- Zero-shot translation between seen language pairs

```python
from transformers import MBartForConditionalGeneration, MBart50TokenizerFast
model = MBartForConditionalGeneration.from_pretrained("facebook/mbart-large-50-many-to-many-mmt")
tokenizer = MBart50TokenizerFast.from_pretrained("facebook/mbart-large-50-many-to-many-mmt")

# Translate Myanmar -> English
tokenizer.src_lang = "my_MM"
encoded = tokenizer("␣␣", return_tensors="pt")
generated = model.generate(**encoded, forced_bos_token_id=tokenizer.lang_code_to_id["en_XX"])
print(tokenizer.batch_decode(generated, skip_special_tokens=True))
# Output: "I love Myanmar"
```

Fine-tuning on custom data

Replace the model's fine-tuning loop with your own parallel Myanmar–target corpus. Use Seq2SeqTrainer from HuggingFace Transformers.

OPUS-MT: Pre-built Translation Models

OPUS-MT (Helsinki-NLP)

OPUS-MT provides hundreds of pre-trained MT models based on Marian NMT, trained on the OPUS parallel corpus collection.

- Available for 500+ language pairs
- Free to use via HuggingFace
- Models for Myanmar–English exist!
- Good starting point for fine-tuning
- Browse all OPUS-MT models at: `https://huggingface.co/Helsinki-NLP`

```python
from transformers import MarianMTModel, MarianTokenizer

model_name = "Helsinki-NLP/opus-mt-my-en"  # Myanmar to English
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)

text = "          ␣                                "
inputs = tokenizer([text], return_tensors="pt", padding=True)
translated = model.generate(**inputs)
print(tokenizer.batch_decode(translated, skip_special_tokens=True))
```

GPT-4 and LLMs as MT Systems

LLMs as Zero-shot Translators

GPT-4 and similar LLMs achieve competitive MT quality through:

- **Massive pretraining** on multilingual web text
- **Instruction following**: tell it to translate without examples
- **In-context learning**: few-shot prompts with examples

Prompting Strategies for MT:

```
# Zero-shot
prompt = "Translate to Myanmar:\n{source}"

# Few-shot
prompt = """Translate English to Myanmar:
Source: Hello, how are you?
Target:                    ?

Source: I love machine translation.
Target: """

# Chain-of-thought
prompt = "Think step by step, then translate...
    "
```

LLM MT Limitations

- **Cost**: API calls expensive at scale
- **Latency**: slower than specialized MT
- **Hallucination**: may add/omit content
- **Low-resource**: poor for rare languages
- **No control**: can't enforce terminology
- **Privacy**: data sent to external API

MT Post-Editing: Human + Machine

Computer-Assisted Translation (CAT)

Modern translators use MT as a **first draft** and **post-edit**:

1. MT system produces a translation
2. Human post-editor corrects errors
3. **Faster** than translating from scratch (studies show 20–50% productivity gain)

Post-editing effort levels:

- **Light PE**: fix only critical errors
- **Full PE**: produce publication-quality output
- **MTPE rate**: typically 0.4–0.6 $\times$ full translation rate

Tools for post-editing:

- **SDL Trados**, **memoQ**, **OmegaT**
- **Moses Integration**: export MT output to TM format

Future Direction

Interactive MT: system adapts in real-time based on post-editor's corrections, using active learning.

Myanmar application: professional translators assisted by Moses/Marian for faster ethnic language document translation.

GNMTv2: Google's Industrial NMT (2016)

Wu et al. (2016) — "Google's Neural Machine Translation System"

GNMT is Google's first production-quality NMT system, deployed in 2016:

- **8-layer deep LSTM** encoder (bidirectional first layer)
- **8-layer deep LSTM** decoder with attention
- **Residual connections** between LSTM layers (key for depth)
- **Word-piece model** (precursor to BPE/SentencePiece)
- **Model parallelism**: split across multiple GPUs
- First deployed for English-French, English-Chinese

Impact

GNMT reduced translation errors by **55–85%** compared to Google's previous phrase-based SMT system. Marked the beginning of the NMT era in commercial deployment. The paper demonstrated that NMT was production-ready.

ConvSeq2Seq: Convolutional MT

Gehring et al. (2017) — Facebook Research

Convolutional Seq2Seq: replace RNNs with **CNNs** for MT.

- **Fully convolutional**: encoder and decoder use stacked convolutions
- **Multi-hop attention**: attention computed from decoder CNN states
- **Gated linear units (GLU)**: activation: $X \odot \sigma(Y)$
- Much faster than RNN (full parallelization within a layer)
- Slightly outperformed RNN-based NMT at the time

Historical Significance

ConvSeq2Seq showed that **sequence modeling does not require RNNs**. This paved the way for the Transformer's full removal of recurrence (2017). The race was: LSTM (2014) $\rightarrow$ CNN (2017) $\rightarrow$ Transformer (2017).

Practical: Setting Up NMT Environment

Step-by-step environment setup for students:

Option A: OpenNMT-py

```
# Install
pip install OpenNMT-py
pip install sentencepiece

# Quick start with Transformer
onmt_train -config config.yaml

# config.yaml:
# data:
#   corpus_1:
#     path_src: train.my
#     path_tgt: train.en
# train_steps: 50000
# valid_steps: 5000
# save_model: model/my-en
# encoder_type: transformer
# decoder_type: transformer
```

Option B: HuggingFace Transformers

```
pip install transformers datasets sacrebleu

# Fine-tune NLLB or mBART on
# your Myanmar corpus:
from transformers import (
    AutoModelForSeq2SeqLM,
    AutoTokenizer,
    Seq2SeqTrainer,
    Seq2SeqTrainingArguments
)
# Load NLLB-200-distilled-600M
# Add your parallel corpus
# Train with Seq2SeqTrainer
```

Practical: Setting Up NMT Environment

Minimum Hardware

For training from scratch: 1 GPU with 8GB+ VRAM (e.g., GTX 1080, RTX 3080).
For fine-tuning NLLB-600M: 1 GPU with 16GB (A100 preferred), or Google Colab Pro.
For CPU-only: use small models (JoeyNMT) with small data.

Domain Adaptation in MT

Challenge

A general-purpose MT model (trained on news/web data) performs poorly on **specialized domains**: medical, legal, technical, religious.

Domain Adaptation Techniques:

1. **Fine-tuning**: continue training on in-domain data (most common, works well)
2. **Data selection**: select in-domain-like data from general corpus (Moore-Lewis criterion)
3. **Mixed fine-tuning**: combine general + in-domain
4. **Adapter layers**: add small adapter modules without touching pretrained weights
5. **Prompt-based**: for LLMs, add domain context in the prompt

Domain Adaptation in MT

Myanmar Medical Domain

General model may translate medical terms wrong.

Solution: collect Myanmar medical bilingual data (drug descriptions, clinical notes) and fine-tune NLLB-600M for 1k steps.

Even 500–1000 in-domain sentence pairs can give large improvement.